

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНЫМ РАБОТАМ № 1-5
«Алгоритмы на графах»
Вариант 54
по дисциплине Теория графов

Студент,
группы 5130201/40003

_____ Четвергов И.С

Преподаватель

_____ Востров А.В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Математическое описание	6
1.1 Граф и его представление	6
1.2 Распределение Рэлея–Райса	6
1.3 Генерация связного ациклического графа по степеням вершин	7
1.4 Эксцентриситет, центр и диаметральные вершины	8
1.5 Метод Шимбелла	8
1.6 Подсчёт маршрутов	9
1.7 Обход рёбер в ширину	9
1.8 Алгоритм Дейкстры	9
1.9 Определение сети	10
1.10 Поток в сети	10
1.11 Алгоритм Форда–Фалкерсона	10
1.12 Нахождение заданного потока минимальной стоимости	11
1.13 Теорема Кирхгофа	12
1.14 Алгоритм Борувки	12
1.15 Код Прюфера	13
1.16 Минимальная раскраска графа	13
1.17 Эйлеров граф	14
1.18 Достройка графа до эйлера	14
1.19 Алгоритм Иерхольцера	15
1.20 Фундаментальная система разрезов	15
1.21 Симметрическая разность разрезов	15
2 Особенности реализации	17
2.1 Общий поток выполнения	17
2.2 Модуль common	17
2.2.1 GraphBase: шаблонный граф	17
2.2.2 Graph (DAG)	18
2.2.3 Generator: генерация графов	20
2.2.4 CollectionUtils: шаблонные утилиты	23
2.2.5 FileHandler: сохранение данных	24
2.2.6 Visualizer: вызов Python	24
2.3 Модуль lab1	24
2.3.1 GraphMetrics: эксцентриситеты	24
2.3.2 ShimbellMethod: матричная степень	25
2.3.3 PathCounter: подсчёт маршрутов	27
2.4 Модуль lab2	28
2.4.1 BFS: обход рёбер	28
2.4.2 Dijkstra: кратчайшие пути	29
2.4.3 Comparator: сравнение алгоритмов	29

2.5	Модуль lab3	30
2.5.1	FlowNetwork: сеть потоков	30
2.5.2	MaxFlow: алгоритм Форда–Фалкерсона	31
2.5.3	MinCostFlow: поток минимальной стоимости	32
2.5.4	PathUtils: вспомогательные операции на путях	33
2.6	Модуль lab4	33
2.6.1	KirchhoffTheorem: подсчёт числа остовных деревьев	33
2.6.2	Boruvka: алгоритм Борушки	34
2.6.3	PruferCode: кодирование и декодирование кода Прюфера	35
2.6.4	GraphCombinatorics: минимальная раскраска графа	37
2.7	Модуль lab5	39
2.7.1	EulerianCycle: проверка и нахождение эйлера цикла	39
2.7.2	CutSystem: фундаментальная система разрезов	40
2.8	Визуализация: Python-скрипты	40
2.9	Веб-интерфейс	41
3	Результаты работы программы	42
3.1	Лабораторная работа 1	42
3.1.1	Генерация графа по распределению Рэлея–Райса	42
3.1.2	Матрицы смежности и весов	43
3.1.3	Метод Шимбелла	43
3.1.4	Подсчёт всех маршрутов	44
3.1.5	Метрики графа: эксцентриситеты, центр, диаметр	44
3.2	Лабораторная работа 2	46
3.2.1	Обход рёбер в ширину	46
3.2.2	Алгоритм Дейкстры	47
3.2.3	Сравнение алгоритмов	47
3.3	Лабораторная работа 3	48
3.3.1	Формирование сети потоков	48
3.3.2	Максимальный поток	49
3.3.3	Поток минимальной стоимости	49
3.4	Лабораторная работа 4	51
3.4.1	Матричная теорема Кирхгофа	52
3.4.2	Минимальный остов алгоритмом Борушки	52
3.4.3	Кодирование и декодирование кода Прюфера	52
3.4.4	Минимальная раскраска графа	53
3.5	Лабораторная работа 5	54
3.5.1	Проверка эйлеровости и построение эйлера цикла	54
3.5.2	Фундаментальная система разрезов	54
3.5.3	Симметрическая разность подмножества разрезов	56
	Заключение	58
	Список литературы	60

Введение

Задание лабораторной работы 1.

1. Сформировать случайным образом связный ациклический граф в соответствии с заданным распределением (параметры распределения задаются как константы и подбираются экспериментально) с введенным пользователем количеством вершин. Распределение брать из справочника Вадзинского (есть в списке литературы): либо реализовать блок-схему, либо генерирующую формулу (в справочнике дан один из этих вариантов). Генерация происходит по степеням вершин. В зависимости от дальнейших заданий будет использовать ориентированный или неориентированный граф. Можно реализовывать в консоли.
2. Посчитать эксцентриситеты вершин, выделить центр графа и диаметральные вершины.
3. Реализовать метод Шимбелла для сгенерированной с помощью заданного распределения весовой матрицы (пользователь вводит количество ребер), в ответе представить минимальный и/или максимальный путь (в виде матрицы). Весовая матрица генерируется также в соответствии с заданным распределением. При генерации необходимо учесть генерацию только положительных значений, только отрицательных значений и смешанную (на выбор пользователя).
4. Определить возможность построения маршрута от одной заданной точки до другой (вершины вводит пользователь) и указывать количество таких маршрутов.
5. Реализовать меню для управления всеми пунктами.

Задание лабораторной работы 2.

1. Для заданных графов (случайно сгенерированных в предыдущей работе) выполнить обход рёбер графа поиском в ширину.
2. Сгенерировать весовую матрицу (положительную или с отрицательными весами — выбирает пользователь) и найти кратчайший путь для двух выбранных пользователем вершин. В результате выводится не только расстояние, но и сам путь в виде последовательности вершин. Реализовать классический алгоритм Дейкстры с выводом вектора расстояний.
3. Сравнить скорости работы реализованных алгоритмов по количеству итераций.

Задание лабораторной работы 3.

1. На основе сгенерированного ориентированного графа получить случайным образом матрицы пропускных способностей и стоимости.
2. Для полученного графа найти максимальный поток по алгоритму Форда–Фалкерсона.
3. Вычислить поток минимальной стоимости, где в качестве целевой величины потока берётся $\lfloor 2/3 \cdot \max \rfloor$, $\max$ — максимальный поток. Использовать алгоритм Беллмана–Форда для поиска кратчайшего пути по стоимости в остаточной сети.

Задание лабораторной работы 4.

1. Для заданных неориентированных графов найти число остовных деревьев, используя матричную теорему Кирхгофа.
2. Построить минимальный по весу остов алгоритмом Борувки. Полученный остов закодировать кодом Прюфера и декодировать его; веса сохраняются при кодировании.
3. Реализовать на исходном графе и полученном остове (по выбору пользователя) алгоритм нахождения минимальной раскраски графа.

Задание лабораторной работы 5.

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) проверить, является ли граф эйлеровым. Если нет, то модифицировать граф (логировать, что изменено). Построить эйлеров цикл.
2. Для заданных неориентированных графов (случайно сгенерированных в первой работе) построить кратчайший остов (алгоритмом из четвертой работы). На основе данного остова получить фундаментальную систему разрезов (вывести ее на экран), получение всех остальных разрезов на основе фундаментальных с помощью операции симметрической разности (выбранные разрезы вводит пользователь).

Реализация выполнена на C++20. Проект разбит на статические библиотеки, собираемые через CMake.

1. Математическое описание

1.1. Граф и его представление

Граф — пара $G = (V, E)$, где V — конечное непустое множество вершин, $E \subseteq V \times V$ — множество рёбер. Граф называется ориентированным, если каждое ребро — упорядоченная пара, и неориентированным в противном случае. Ациклический граф не содержит циклов; связный ациклический неориентированный граф является деревом, ориентированный — DAG (directed acyclic graph).

Взвешенный граф задаёт функцию весов $w : E \rightarrow \mathbb{R}$, ставящую каждому ребру вещественное число. В данной работе веса генерируются по распределению Рэлея–Райса с параметрами, задаваемыми пользователем.

1.2. Распределение Рэлея–Райса

Согласно справочнику Вадзинского (п. 3.16), обобщённое распределение Рэлея (распределение Рэлея–Райса) задаётся плотностью вероятности:

$$f(z) = \frac{z}{a^2} e^{-(z^2+h^2)/(2a^2)} I_0\left(\frac{zh}{a^2}\right), \quad z \geq 0,$$

где $a > 0$ — параметр масштаба, $h \geq 0$ — параметр формы, I_0 — модифицированная функция Бесселя нулевого порядка.

Для генерации случайных чисел справочник приводит следующий алгоритм, если $u_1, u_2 \sim \mathcal{N}(0, 1)$ — независимые стандартные нормальные значения, то

$$z = \sqrt{(h + au_1)^2 + (au_2)^2}.$$

На рис. 1 слева указан график из справочника, формулы которого были описаны выше, справа находится график распределения его программной реализации при $n = 1000$, равному числу вершин графа.

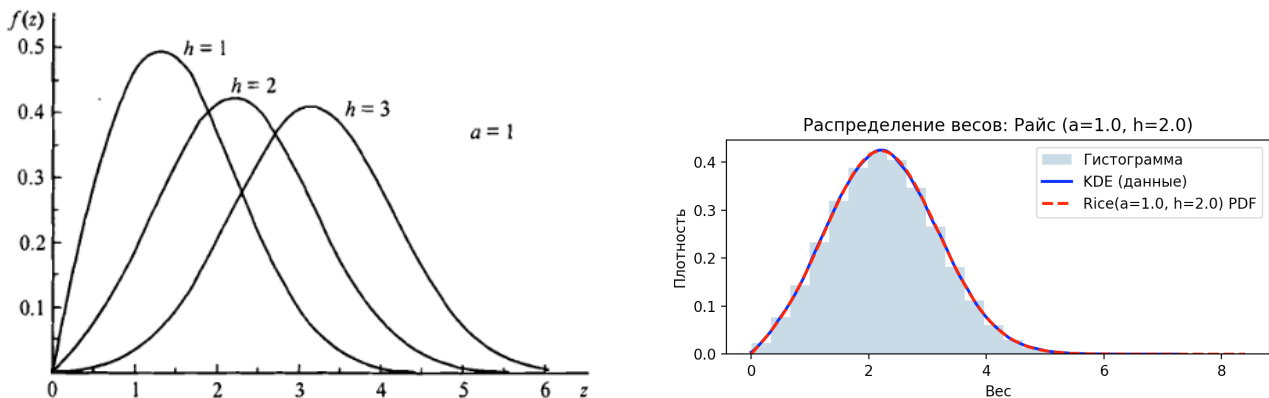


Рис. 1: График из справочника и распределение реализации

Как видно из графиков, полученное распределение совпало с эталонным.

1.3. Генерация связного ациклического графа по степеням вершин

Пользователь задаёт число вершин n и параметры распределения a, h . Число рёбер m не вводится напрямую.

Шаг 1. Для каждой вершины $i \in \{0, \dots, n-1\}$ независимо сэмплируется значение по формуле Вадзинского:

$$x_i = \sqrt{(h + au_{i,1})^2 + (au_{i,2})^2}, \quad u_{i,1}, u_{i,2} \sim \mathcal{N}(0, 1).$$

Вектор $\mathbf{x} = (x_0, \dots, x_{n-1})$ трактуется как ненормированные веса вершин. Для перевода в вектор вероятностей применяется функция softmax со стабилизацией:

$$\text{softmax}(\mathbf{x})_i = p_i = \frac{e^{x_i - x_{\max}}}{\sum_{j=0}^{n-1} e^{x_j - x_{\max}}}, \quad \sum_{i=0}^{n-1} p_i = 1.$$

Шаг 2. Итоговое число рёбер m сэмплируется из допустимого диапазона $[n-1, n(n-1)/2]$, где $n-1$ — минимум (остовное дерево), $n(n-1)/2$ — максимум (полный граф).

Для каждой вершины i определяется максимально допустимая степень в топологическом порядке:

$$\text{capacity}_i = n - 1 - i.$$

Взвешенная доля вершины i : $q_i = p_i \cdot \text{capacity}_i$. Вектор взвешенных долей масштабируется к сумме m :

$$q_i \leftarrow q_i \cdot \frac{m}{\sum_j q_j}.$$

Целевая степень вершины: $d_i = \lfloor q_i \rfloor$. Если после округления $\sum_i d_i < m$, недостающие единицы последовательно прибавляются к вершинам с наибольшей дробной частью $q_i - \lfloor q_i \rfloor$ при условии $d_i < \text{capacity}_i$.

Шаг 3. Граф строится в два прохода. В первом формируется остовное дерево: для каждой вершины $i \geq 1$ предок выбирается равномерно случайно из множества вершин $j < i$, для которых остаток целевой степени $d_j > 0$; если таких вершин нет — равномерно из $\{0, \dots, i-1\}$. После добавления ребра (j, i) остаток целевой степени предка уменьшается: $d_j \leftarrow d_j - 1$. Построение по индексам $j < i$ гарантирует связность и ацикличность.

Во втором проходе для каждой вершины i с $d_i > 0$ формируется множество допустимых соседей $\{j \mid j > i, (i, j) \notin E\}$, из которого случайно без возвращения выбираются $\min(d_i, |\text{допустимых}|)$ вершин и добавляются соответствующие рёбра.

Веса рёбер сэмплируются по той же формуле Вадзинского.

1.4. Эксцентриситет, центр и диаметральные вершины

Для связного невзвешенного графа $G = (V, E)$ определим кратчайшее расстояние $d(u, v)$ как суммарное расстояние минимального пути между u и v . Эксцентриситет вершины — максимальное расстояние до остальных вершин:

$$\varepsilon(v) = \max_{u \in V} d(v, u).$$

На основе эксцентриситетов вводятся глобальные характеристики графа:

$$\text{rad}(G) = \min_{v \in V} \varepsilon(v), \quad \text{diam}(G) = \max_{v \in V} \varepsilon(v).$$

Радиус графа — множество вершин с минимальным эксцентриситетом:

$$C(G) = \{v \mid \varepsilon(v) = \text{rad}(G)\}$$

Диаметральные вершины — с максимальным:

$$D(G) = \{v \mid \varepsilon(v) = \text{diam}(G)\}$$

Расстояния вычисляются алгоритмом обхода в ширину (*BFS*), запускаемым из каждой вершины, алгоритмическая сложность линейная ($O(V + E)$) для одного прохода, а итоговая сложность квадратична $O(V(V + E))$.

1.5. Метод Шимбелла

Пусть A — матрица весов графа размера $n \times n$: элемент a_{ij} равен весу ребра (i, j) , если оно существует, иначе считается неопределённым (`nullopt` в реализации). Метод Шимбелла позволяет найти длины кратчайших и наидлиннейших путей из ровно k рёбер для всех пар вершин одновременно.

Метод основан на переопределении двух алгебраических операций. Операция «умножения» двух величин a и b заменяется их алгебраической суммой:

$$a \cdot b = b \cdot a \Rightarrow a + b = b + a, \quad a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0.$$

Операция «сложения» заменяется выбором минимального или максимального из двух элементов:

$$a + b = b + a = \min(\max)\{a, b\},$$

при этом нулевые (неопределённые) элементы игнорируются — минимум или максимум выбирается из ненулевых значений.

На основе этих операций определяются два тропических матричных умножения. Для операции $\otimes_{\min}$:

$$(A \otimes_{\min} B)_{ij} = \min_k (a_{ik} + b_{kj}),$$

и аналогично $\otimes_{\max}$ с заменой $\min$ на $\max$. В общем виде элемент матрицы $A^{(2)}$, полученной возведением A в степень 2, вычисляется как:

$$\omega_{ij}^{(2)} = \min(\max) \left\{ \omega_{i1}^{(1)} + \omega_{1j}^{(1)}, \omega_{i2}^{(1)} + \omega_{2j}^{(1)}, \dots, \omega_{in}^{(1)} + \omega_{nj}^{(1)} \right\}.$$

Матрица $A^{(k)}$, вычисленная последовательным умножением начиная с исходной матрицы A , содержит длины путей из ровно k рёбер для всех пар вершин. Временная сложность: $O(k \cdot n^3)$, пространственная: $O(n^2)$.

1.6. Подсчёт маршрутов

Маршрут от вершины s до вершины t — простой путь, не повторяющий вершины. Все маршруты находятся методом обхода с возвратом (backtracking): рекурсивный DFS с маской посещённых вершин. При достижении целевой вершины текущий путь копируется в результат; при выходе из рекурсии вершина снимается с отметки, что обеспечивает полный перебор всех вариантов. Сложность в худшем случае — $O(n!)$, пространственная сложность $O(V)$.

1.7. Обход рёбер в ширину

Обход в ширину (*BFS*) из вершины s посещает все вершины графа. В основе алгоритма лежит очередь Q и множество посещённых вершин *visited*. На каждом шаге вершина u извлекается из головы очереди; все инцидентные рёбра (u, v) обходятся, и каждая непосещённая вершина v помечается и добавляется в хвост очереди Q .

Алгоритмическая сложность линейная $O(V + E)$ и пространственная сложность $O(V)$. Результатом обхода алгоритма является последовательность рёбер в порядке их первого посещения и вектор расстояний $d[v]$ — минимальное число рёбер от s до v .

1.8. Алгоритм Дейкстры

Алгоритм Дейкстры находит кратчайшие пути от источника s до всех вершин во взвешенном графе с неотрицательными весами рёбер. Поддерживается вектор расстояний $d[v]$, инициализированный $+\infty$ для всех вершин кроме s : $d[s] = 0$. На каждом шаге из приоритетной очереди (min-heap) извлекается вершина u с минимальным $d[u]$; для каждого ребра (u, v) с весом $w(u, v)$ выполняется релаксация:

$$d[v] \leftarrow \min(d[v], d[u] + w(u, v)).$$

Если значение обновилось — пара $(d[v], v)$ добавляется в очередь, а предшественник $p[v] \leftarrow u$ записывается для восстановления пути.

Кратчайший путь до целевой вершины t восстанавливается обратным проходом по массиву p : начиная с t , следуем $t \rightarrow p[t] \rightarrow p[p[t]] \rightarrow \dots \rightarrow s$ и разворачиваем последовательность.

Алгоритмическая сложность на бинарной куче: $O((V+E) \log V)$, пространственная сложность $O(V)$.

1.9. Определение сети

Пусть $G = (V, E)$ — орграф с одним истоком $s \in V$ и одним стоком $t \in V$, где $s \neq t$. **Сетью** $S = (G; c)$ называется орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное действительное число $c(e)$, называемое **пропускной способностью** этой дуги e .

1.10. Поток в сети

Потоком в сети S называется функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, для которой выполняются два условия.

Первое — ограничение пропускной способности: для каждой дуги $e \in E$

$$f(e) \leq c(e).$$

Второе — закон сохранения потока: для каждой вершины $v \in V$, $v \neq s, t$,

$$D_f(v) = \sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) = 0,$$

где $A(v)$ — множество дуг, исходящих из v , $B(v)$ — множество дуг, входящих в v .

Величиной потока f называется число

$$p_f = \sum_{e \in A(s)} f(e).$$

Число $w(f) \stackrel{\text{def}}{=} \text{div}(f, s)$ также называется величиной потока f . Дуга (u, v) называется **насыщенной**, если $f(u, v) = c(u, v)$.

1.11. Алгоритм Форда–Фалкерсона

Теорема (Л. Р. Форд, Д. Р. Фалкерсон, 1962). *Величина максимального потока $p_{\max}$ в сети S совпадает с минимальной пропускной способностью $r_{\min}$ её разрезом.*

Алгоритм итеративно находит увеличивающий путь от истока к стоку в остаточной сети — путь, по которому остаточная пропускная способность каждого

ребра строго положительна. Величина дополнительного потока по найденному пути:

$$\Delta f = \min_{(u,v) \in P} r(u, v).$$

Для каждого ребра (u, v) пути P поток обновляется:

$$f(u, v) \leftarrow f(u, v) + \Delta f, \quad f(v, u) \leftarrow f(v, u) - \Delta f.$$

Алгоритм завершается, когда увеличивающий путь не существует. При этом для корректной работы алгоритма требуется, чтобы пропускные способности (и стоимости) рёбер были неотрицательными. В реализации поиск пути выполняется через BFS, что гарантирует сложность $O(V \cdot E^2)$.

1.12. Нахождение заданного потока минимальной стоимости

Рассмотрим задачу определения потока заданной величины θ от s к t в сети $G = (S, U, \Omega, D)$, в которой каждая дуга (x_i, x_j) характеризуется пропускной способностью $c(x_i, x_j) \in \Omega$ и неотрицательной стоимостью $d(x_i, x_j) \in D$ пересылки единичного потока из x_i в x_j вдоль дуги (x_i, x_j) .

Если $\theta > \varphi_{\max}$, где $\varphi_{\max}$ — величина максимального потока в сети G от s к t , то решения не существует. При $\theta \leq \varphi_{\max}$ может быть определено несколько потоков величины θ . Математическая модель задачи: минимизировать

$$\sum_{(x_i, x_j) \in U} d(x_i, x_j) \cdot \varphi(x_i, x_j),$$

где $\varphi(x_i, x_j)$ — поток по дуге (x_i, x_j) , при ограничениях:

1. $0 \leq \varphi(x_i, x_j) \leq c(x_i, x_j)$ для всех $(x_i, x_j) \in U$;
2. для начальной вершины $s \in S$ (уравнение истока):

$$\sum_{x_j \in S} \varphi(s, x_j) - \sum_{x_j \in S} \varphi(x_j, s) = \theta$$

3. для конечной вершины $t \in S$ (уравнение стока):

$$\sum_{x_j \in S} \varphi(t, x_j) - \sum_{x_j \in S} \varphi(x_j, t) = -\theta$$

4. для любой другой вершины $x_j \neq s, t$ (условие сохранения потока):

$$\sum_{x_i \in S} \varphi(x_j, x_i) - \sum_{x_i \in S} \varphi(x_i, x_j) = 0$$

1.13. Теорема Кирхгофа

Пусть $G = (V, E)$ — связный неориентированный граф без петель и кратных рёбер, $|V| = n$. Матрица Кирхгофа $B = B(G) = (b_{ij})_{n \times n}$ определяется как:

$$B = D - A(G),$$

где $D = \text{diag}(\deg v_1, \dots, \deg v_n)$ — диагональная матрица степеней вершин, $A(G)$ — матрица смежности графа G . Явно:

$$b_{ij} = \begin{cases} \deg v_i, & i = j, \\ -1, & i \neq j, v_i \sim v_j, \\ 0, & i \neq j, v_i \not\sim v_j. \end{cases}$$

Сумма элементов каждой строки (и каждого столбца) матрицы B равна нулю, поэтому $\det B = 0$ и $\lambda_i = 0$ — собственное значение B . Для связного графа кратность нулевого собственного значения равна единице, остальные $n - 1$ собственных значений строго положительны.

Лемма Все алгебраические дополнения элементов матрицы Кирхгофа равны между собой.

Теорема (матричная теорема Кирхгофа). Число различных остовных деревьев связного графа G равно любому определителю его матрицы Кирхгофа:

$$\tau(G) = \det B_{ii},$$

где B_{ii} — матрица B с удалённой i -й строкой и i -м столбцом.

Алгоритмическая сложность: $O(n^3)$, пространственная: $O(n^2)$.

1.14. Алгоритм Борувки

Алгоритм Борувки — алгоритм построения минимального остовного дерева во взвешенном связном неориентированном графе $G = (V, E)$ с функцией весов $w : E \rightarrow \mathbb{R}$. Впервые опубликован О. Борувкой в 1926 году.

Минимальное остовное дерево (МОД) — остовное дерево $T \subseteq E$ с минимальным суммарным весом $\sum_{e \in T} w(e)$.

Описание алгоритма.

1. Изначально каждая вершина графа — тривиальное дерево (компонента из одной вершины); множество рёбер остова $T = \emptyset$.
2. Для каждой компоненты C_k находится минимальное по весу ребро, инцидентное ей и соединяющее её с другой компонентой. При равенстве весов среди кандидатов выбирается ребро с наименьшим индексом, что исключает добавление цикла. Все найденные рёбра добавляются в T ; компоненты, соединённые этими рёбрами, объединяются.

3. Шаг 2 повторяется, пока в графе не останется одна компонента.

Теорема. Алгоритм Борувки строит минимальное остовное дерево.

Алгоритмическая сложность: $O(E \log V)$, пространственная: $O(V + E)$.

1.15. Код Прюфера

Кодирование. Пусть $T = (V, E_T)$ — дерево с вершинами $V = \{1, \dots, n\}$. Код Прюфера — последовательность $A = (A[1], \dots, A[n-2])$, строящаяся следующим образом: на каждом из $n-2$ шагов выбирается висячая вершина v с минимальным номером ($\deg v = 1$), в код записывается её единственный сосед $\Gamma(v)$, после чего вершина v удаляется из дерева:

$$A[i] = \Gamma(\min\{k \in V \mid \deg(k) = 1\}), \quad V \leftarrow V \setminus \{v\}, \quad i = 1, \dots, n-2.$$

В конце остаются ровно две вершины; информация о последнем ребре не хранится, поскольку восстанавливается однозначно.

Код Прюфера — наиболее экономное по памяти представление помеченного дерева: последовательность из $n-2$ чисел кодирует n^{n-2} различных помеченных деревьев (формула Кэли).

Декодирование. По коду $A = (A[1], \dots, A[n-2])$ восстанавливается дерево $T = (V, E_T)$. Поддерживается множество «свободных» вершин $B = \{1, \dots, n\}$. На каждом шаге i выбирается наименьшая вершина $v \in B$, не встречающаяся в остатке кода $A[i..n-2]$; добавляется ребро $(v, A[i])$, вершина v удаляется из B :

$$v = \min\{k \in B \mid \forall j \geq i : k \neq A[j]\}, \quad E_T \leftarrow E_T \cup \{(v, A[i])\}, \quad B \leftarrow B \setminus \{v\}.$$

После $n-2$ шагов в B остаются ровно две вершины — они соединяются последним ребром. Алгоритмическая сложность кодирования и декодирования: $O(n \log n)$, пространственная: $O(n)$.

Веса рёбер МОД сохраняются при кодировании: каждому ребру $(u, v) \in E_T$ сопоставляется вес $w(u, v)$; при декодировании веса восстанавливаются по соответствию между рёбрами исходного остова и рёбрами, добавляемыми на каждом шаге декодирования.

1.16. Минимальная раскраска графа

Правильная раскраска графа $G = (V, E)$ — отображение $c : V \rightarrow \{1, \dots, k\}$ такое, что для любого ребра $(u, v) \in E$ выполняется $c(u) \neq c(v)$. Минимальное число цветов, при котором существует правильная раскраска, называется **хроматическим числом** $\chi(G)$.

Нахождение $\chi(G)$ является NP-трудной задачей в общем случае. В данной работе реализован точный алгоритм на основе полного перебора с отсечениями (backtracking): вершины обходятся в фиксированном порядке; каждой вершине

последовательно назначается наименьший допустимый цвет — такой, которым не окрашен ни один из уже обработанных соседей. Если при некотором числе цветов k раскраска найдена, алгоритм повторяется для $k - 1$ цветов; процедура продолжается до тех пор, пока раскраска при $k - 1$ цветах не окажется невозможной.

Алгоритмическая сложность в худшем случае: $O(k^V)$, пространственная: $O(V)$.

1.17. Эйлеров граф

Граф $G = (V, E)$ называется *эйлеровым*, если в нём существует замкнутый маршрут, проходящий по каждому ребру ровно один раз — *эйлеров цикл*. Граф называется *полуэйлеровым*, если такой маршрут существует, но не является замкнутым — *эйлеров путь*.

Теорема (Эйлер, 1736). *Связный неориентированный граф является эйлеровым тогда и только тогда, когда степень каждой вершины чётна:*

$$\forall v \in V : \deg(v) \equiv 0 \pmod{2}.$$

Граф является полуэйлеровым тогда и только тогда, когда он связан и имеет ровно две вершины нечётной степени; эти вершины служат началом и концом эйлерова пути.

Множество вершин нечётной степени обозначим:

$$\text{Odd}(G) = \{v \in V \mid \deg(v) \equiv 1 \pmod{2}\}.$$

Тогда: G эйлеров $\Leftrightarrow \text{Odd}(G) = \emptyset$; G полуэйлеров $\Leftrightarrow |\text{Odd}(G)| = 2$.

1.18. Достройка графа до эйлерова

Если $\text{Odd}(G) \neq \emptyset$, граф модифицируется добавлением рёбер. Процедура состоит из двух этапов.

Этап 1. Если граф несвязен, ненулевые компоненты связности $C_1, C_2, \dots, C_p$ соединяются цепочкой дополнительных рёбер: для каждого $k = 2, \dots, p$ добавляется ребро (u_{k-1}, u_k) , где $u_k \in C_k$ — произвольный представитель компоненты. После этого граф становится связным.

Этап 2. Пока $|\text{Odd}(G)| > 2$, вершины из $\text{Odd}(G)$ разбиваются на пары и между каждой парой добавляется ребро. Каждое добавление уменьшает $|\text{Odd}(G)|$ ровно на два: оба конца нового ребра меняют чётность степени. Если $|\text{Odd}(G)| = 2$, добавляется одно замыкающее ребро, причем если такое ребро уже существует в графе, то для замыкания полуэйлерова графа до эйлерова оно не дублируется, а извлекается (удаляется) из системы — граф становится эйлеровым.

1.19. Алгоритм Иерхольцера

Алгоритм Иерхольцера находит эйлеров цикл (или путь) в графе, удовлетворяющем условию теоремы Эйлера. Используется итеративная реализация на основе стека.

Для старта выбирается вершина $s \in \text{Odd}(G)$ при полуэйлеровом графе, иначе — произвольная вершина с ненулевой степенью. Поддерживается стек stk , вектор использованных рёбер $used : E \rightarrow \{0, 1\}$ и указатели $ptr[u]$ — индекс следующего непросмотренного ребра вершины u .

На каждом шаге рассматривается вершина $u = stk.top()$. Если существует неиспользованное ребро (u, v) — оно помечается ($used[id] \leftarrow 1$) и v помещается на стек. Если таких рёбер нет — u снимается со стека и дописывается в маршрут π . По завершении π разворачивается.

Корректность проверяется двумя условиями:

$$|\pi| = |E| + 1$$

Алгоритмическая сложность: $O(V + E)$, пространственная: $O(V + E)$.

1.20. Фундаментальная система разрезов

Пусть $T = (V, E_T)$ — остовное дерево графа $G = (V, E)$. Удаление ребра $e \in E_T$ разбивает T на два поддерева с множествами вершин A и $B = V \setminus A$. *Фундаментальный разрез* ребра e — множество рёбер исходного графа, пересекающих это разбиение:

$$S(e) = \{(u, v) \in E \mid (u \in A \wedge v \in B) \vee (u \in B \wedge v \in A)\}.$$

Заметим, что $e \in S(e)$ всегда, поскольку ребро остова само пересекает разбиение.

Совокупность фундаментальных разрезов по всем рёбрам остова образует *фундаментальную систему разрезов* (ФСР):

$$\mathcal{F}(T) = \{S(e) \mid e \in E_T\}.$$

$|\mathcal{F}(T)| = |E_T| = |V| - 1$. Разрезы из $\mathcal{F}(T)$ линейно независимы и порождают пространство разрезов графа размерности $|V| - 1$.

1.21. Симметрическая разность разрезов

Произвольный разрез графа выражается через фундаментальные с помощью операции *симметрической разности*:

$$S_1 \triangle S_2 = (S_1 \setminus S_2) \cup (S_2 \setminus S_1) = (S_1 \cup S_2) \setminus (S_1 \cap S_2).$$

Операция Δ ассоциативна и коммутативна. Ребро (u, v) входит в симметрическую разность подмножества разрезов тогда и только тогда, когда оно встречается в нечётном числе слагаемых:

$$S(e_{i_1}) \Delta \cdots \Delta S(e_{i_k}) = \left\{ (u, v) \in E \mid \sum_{j=1}^k \mathbf{1}[(u, v) \in S(e_{i_j})] \equiv 1 \pmod{2} \right\}.$$

2. Особенности реализации

2.1. Общий поток выполнения

Проект собирается как набор статических библиотек через CMake. Точка входа — единый исполняемый файл `graph_main`, который инициализирует `Menu` и запускает цикл ввода. Зависимости между модулями однонаправленные: лабораторные зависят от `common`, `common` не знает ни о каких лабораторных.

Типичный сценарий выполнения выглядит так. Пользователь выбирает пункт меню, `Menu` вызывает соответствующий метод `Runner`'а из нужной лабы. `Runner` читает параметры из консоли, делегирует работу алгоритмическому классу (`Generator`, `ShimbellMethod` и т. д.), затем передаёт результат в `FileHandler` для сохранения и в `Visualizer` для отрисовки. `Visualizer` вызывает Python-скрипт через `system()`, передавая пути к файлам как аргументы командной строки. Python читает данные из `assets/txt/` и сохраняет картинку в `assets/png/`.

```
main.cc -> Menu -> Runner (lab1/lab2/...) -> Generator ->
    -> Algorithms -> FileHandler -> assets/txt/ ->
    -> Visualizer -> Python -> assets/png/
```

Рис. 2: Поток выполнения

2.2. Модуль common

Модуль содержит весь код, который переиспользуется несколькими лабораторными: базовый граф, генератор, визуализатор, файловый обработчик, утилиты и меню. Сборка через CMake в статическую библиотеку `graph_common`.

2.2.1. GraphBase: шаблонный граф

`GraphBase<VertexT, EdgeT>` — базовый шаблонный класс, параметризованный типами вершины и ребра.

```
1 template <VertexType VertexT, EdgeType EdgeT>
2 class GraphBase {
3     unordered_map<uint64_t, unique_ptr<VertexT>> m_vertices_;
4     unordered_map<uint64_t, EdgeT> m_edges_;
5     bool is_directed_ = false;
6     ...
7 };
```

Листинг 1: Объявление `GraphBase` (`GraphBase.h`)

Класс служит основой для `Graph` (`GraphBase<Vertex, EdgeData>`) и `FlowNetwork` (`GraphBase<FlowVertex, FlowEdge>`).

Хранение данных. Вершины и рёбра лежат в хэш-таблицах с ключом $key(from, to) = (from \ll 32) \mid to$. Для неориентированных графов пара нормализуется перед хешированием: $from \leq to$. Это даёт $O(1)$ проверку наличия ребра и автоматически обеспечивает симметрию.

Список инцидентности. Смежные вершины хранятся в каждом объекте вершины в `vector<int> m_neighbors_`. При вызове `addEdge` обновляются и хэш-таблица рёбер, и список соседей. Запрос `neighbors(v)` возвращает этот вектор за $O(\deg v)$. Такая двойная структура — хэш по рёбрам плюс список соседей в вершине — позволяет не выбирать между скоростью обхода и скоростью проверки.

RAII и владение. RAII (*Resource Acquisition Is Initialization*) — идиома C++, при которой ресурс захватывается в конструкторе и освобождается в деструкторе, время жизни ресурса привязано к времени жизни объекта. Вершины хранятся как `unique_ptr<VertexT>` — владение однозначно, удаление происходит автоматически при разрушении контейнера, утечки памяти исключены на уровне типа.

Ребро хранится как значение `EdgeT` — копирование дёшево (структура из двух `int` и `double`). Вершина — указатель, потому что `VertexT` содержит `vector<int> m_neighbors_`, который растёт динамически; хранение по значению в `unordered_map` привело бы к инвалидации ссылок при рехешировании.

Все немодифицирующие методы помечены `[[nodiscard]]` — атрибут стандарта C++17, при котором компилятор предупреждает об игнорировании возвращаемого значения:

```
1 [[nodiscard]] bool hasVertex(int id) const noexcept;
2 [[nodiscard]] bool hasEdge(int from, int to) const;
3 [[nodiscard]] std::optional<EdgeT> getEdge(int from, int to);
4 [[nodiscard]] std::vector<int> vertexIds() const;
5 [[nodiscard]] std::vector<EdgeT> edges() const;
6 [[nodiscard]] std::vector<int> neighbors(int id) const;
7 [[nodiscard]] int degree(int v) const;
```

Листинг 2: Публичный интерфейс GraphBase (GraphBase.h)

Возвращаемые значения с неопределённым результатом (отсутствующее ребро) возвращаются через `std::optional<EdgeT>` — явный способ выразить «может не быть», без исключений и без неявного кода ошибки.

2.2.2. Graph (DAG)

Graph наследует `GraphBase<Vertex, EdgeData>`.

```
1 struct EdgeData {
2     int from{-1};
3     int to{-1};
4     double weight{1.0};
```

```

5
6   EdgeData() = default;
7   EdgeData(int f, int t, double w) : from(f), to(t), weight(w) {}
8   bool operator<(EdgeData const& other) const { return weight <
9       other.weight; }
10
11  class Vertex {
12  public:
13      explicit Vertex(int id) : m_id_(id) {}
14      [[nodiscard]] int id() const noexcept { return m_id_; }
15      void addNeighbor(int neighborId);
16      [[nodiscard]] std::vector<int> neighbors() const;
17  private:
18      int m_id_;
19      std::vector<int> m_neighbors_;
20  };
21
22  class Graph : public GraphBase<Vertex, EdgeData> {
23  public:
24      explicit Graph(bool isDirected = false) : GraphBase(isDirected) {}
25      bool addEdge(int from, int to, double weight);
26      [[nodiscard]] std::optional<double> getEdgeWeight(int from, int to)
27          const;
28      [[nodiscard]] std::vector<std::pair<int, double>> neighbors(int id)
29          const;
30      [[nodiscard]] std::vector<int> getNeighbors(int id) const;
31      void printGraphInfo() const;
32  };

```

Листинг 3: Graph, Vertex, EdgeData (Graph.h)

EdgeData хранит поля `from`, `to` и `weight`. Оператор `<` по весу позволяет сортировать рёбра без дополнительных компараторов — используется в алгоритмах, требующих упорядочения рёбер.

Vertex хранит идентификатор и список смежных вершин `m_neighbors_`. Метод `addNeighbor` вызывается из `Graph::addEdge`: для неориентированного графа смежность добавляется в обоих направлениях — ребро регистрируется в хэш-таблице через `GraphBase::addEdge`, а входящая вершина получает обратную запись через явный вызов `m_vertices_[to]->addNeighbor(from)`.

Graph расширяет интерфейс базового класса тремя методами. `getEdgeWeight(from, to)` возвращает `optional<double>` — вес ребра или `nullopt` если ребра нет. `neighbors(id)` возвращает список смежных вершин с весами `vector<pair<int, double>>`, объединяя данные из списка соседей и хэш-таблицы рёбер за один проход. `getNeighbors(id)` делегирует в `GraphBase::neighbors` и возвращает только идентификаторы — используется там, где вес не нужен.

2.2.3. Generator: генерация графов

Генератор использует шаблонный приватный метод, принимающий лямбды `addEdge` и `genWeight`. Это позволяет переиспользовать один алгоритм структурной генерации для разных типов графов и способов генерации весов. Публичные методы лишь собирают нужные лямбды и передают их в шаблонный метод.

Для управления режимами используются `enum class` — исключают неявное преобразование в `int`:

```
1 enum class EdgeCountDist { Uniform, TruncatedNormal };
2 enum class WeightSign    { Positive, Negative, Mixed };
```

Листинг 4: Перечисления параметров генератора (Generator.h)

Степенная генерация: `generateRiceGraphByDegrees`. Метод строит граф в три шага, связанных через `buildByRice`.

Шаг 1: число рёбер (`sampleEdgeCount`).

Вход: $m_{\min} = n - 1$, $m_{\max} = \frac{n(n-1)}{2}$, режим `EdgeCountDist`.

Выход: `int` — число рёбер m .

Описание: `Uniform` — равномерная выборка из $[m_{\min}, m_{\max}]$. `TruncatedNormal` — нормальное распределение $\mathcal{N}(\mu, \sigma^2)$ с $\mu = (m_{\min} + m_{\max})/2$ и $\sigma = (m_{\max} - m_{\min})/6$, усечённое методом отклонения.

```
1 int Generator::sampleEdgeCount(int minE,
2                               int maxE,
3                               EdgeCountDist dist)
4 {
5     if (minE >= maxE) return minE;
6     if (dist == EdgeCountDist::Uniform)
7         return randomInt(minE, maxE);
8     double mu    = (minE + maxE) / 2.0;
9     double sigma = (maxE - minE) / 6.0;
10    double val;
11
12    do { val = mu + sigma * normal_dist_(rng_); }
13    while (val < minE || val > maxE);
14
15    return static_cast<int>(std::round(val));
16 }
```

Листинг 5: Выбор числа рёбер (Generator.cc)

Шаг 2: целевые степени (`computeDegreesFromRice`).

Вход: n, a, h, m .

Выход: `vector<int>` длины n .

Описание: Для каждой вершины сэмплируется $x_i = \sqrt{(h + au_1)^2 + (au_2)^2}$, $u_1, u_2 \sim \mathcal{N}(0, 1)$. Вектор нормируется через softmax со стабилизацией. Степень

вершины i : $q_i = p_i \cdot \text{cap}_i$, где $\text{cap}_i = n - 1 - i$ — максимальная степень в топологическом порядке. Квоты масштабируются к сумме m , целые части берутся через `floor`, остаток распределяется жадно по убыванию дробных частей.

```

1 vector<int> Generator::computeDegreesFromRice(int n,
2                                             bool isDirected,
3                                             double a,
4                                             double h,
5                                             int totalEdges)
6 {
7     std::vector<double> prob(n);
8     for (auto& v : prob) v = riceWeight(a, h);
9
10    double maxVal = *std::max_element(prob.begin(), prob.end());
11    double sumExp = 0.0;
12    for (auto& v : prob) { v = std::exp(v - maxVal); sumExp += v; }
13    for (auto& v : prob) v /= sumExp;
14
15    std::vector<int> cap(n);
16    for (int i = 0; i < n; ++i) cap[i] = n - 1 - i;
17
18    std::vector<double> quota(n);
19    double sumW = 0.0;
20    for (int i = 0; i < n; ++i) { quota[i] = prob[i] * cap[i]; sumW +=
21        quota[i]; }
22    if (sumW > 0.0)
23        for (auto& q : quota) q *= static_cast<double>(totalEdges) / sumW;
24
25    std::vector<int> degrees(n);
26    int allocated = 0;
27    for (int i = 0; i < n; ++i) {
28        degrees[i] = std::min((int)std::floor(quota[i]), cap[i]);
29        allocated += degrees[i];
30    }
31    int remainder = totalEdges - allocated;
32    std::vector<int> order(n);
33    std::iota(order.begin(), order.end(), 0);
34    std::sort(order.begin(), order.end(), [&](int a, int b) {
35        return (quota[a] - std::floor(quota[a]))
36            > (quota[b] - std::floor(quota[b]));
37    });
38    for (int idx : order) {
39        if (remainder <= 0) break;
40        if (degrees[idx] < cap[idx]) { ++degrees[idx]; --remainder; }
41    }
42    return degrees;
43 }

```

Листинг 6: Вычисление целевых степеней (Generator.cc)

Шаг 3: построение графа (`generateByDegreesTemplate`).

Вход: n , вектор `degrees`, лямбды `addEdge` и `genWeight`, флаг ориентированности.

Выход: `unique_ptr<GraphT>`.

Описание: Первый проход строит остовное дерево: для вершины i предок выбирается из вершин с ненулевым бюджетом степени; если таких нет — случайно. Второй проход подбирает рёбра согласно оставшемуся бюджету. Ациклические свойства гарантируются тем, что рёбра в обоих проходах добавляются только от u к v при $u < v$.

```
1 template <typename GraphT,  
2         typename AddEdgeFunc,  
3         typename WeightGenFunc>  
4 std::unique_ptr<GraphT> Generator::generateByDegreesTemplate(  
5         int numVertices,  
6         std::vector<int> degrees,  
7         AddEdgeFunc addEdge,  
8         WeightGenFunc genWeight,  
9         bool isDirected)  
10 {  
11     auto graph = std::make_unique<GraphT>(isDirected);  
12     for (int i = 0; i < numVertices; ++i) graph->addVertex(i);  
13  
14     for (int i = 1; i < numVertices; ++i) {  
15         std::vector<int> with_budget;  
16         for (int j = 0; j < i; ++j)  
17             if (degrees[j] > 0) with_budget.push_back(j);  
18         int from = with_budget.empty()  
19             ? randomInt(0, i - 1)  
20             : with_budget[randomInt(0, (int)with_budget.size() - 1)];  
21         addEdge(*graph, from, i, genWeight());  
22         if (degrees[from] > 0) --degrees[from];  
23     }  
24  
25     for (int i = 0; i < numVertices - 1; ++i) {  
26         if (degrees[i] <= 0) continue;  
27         std::vector<int> available;  
28         for (int j = i + 1; j < numVertices; ++j)  
29             if (!graph->hasEdge(i, j)) available.push_back(j);  
30         std::shuffle(available.begin(), available.end(), rng_);  
31         int toAdd = std::min(degrees[i], (int)available.size());  
32         for (int k = 0; k < toAdd; ++k)  
33             addEdge(*graph, i, available[k], genWeight());  
34     }  
35     return graph;  
36 }
```

Листинг 7: Построение графа по целевым степеням (Generator.hpp)

Точка входа степенной генерации:

```
1 std::unique_ptr<Graph> Generator::generateRiceGraphByDegrees(  
2         int numVertices,  
3         bool isDirected,  
4         double a,  
5         double h,  
6         EdgeCountDist dist,
```

```

7         WeightSign sign)
8     {
9         auto gen_weight = [this, a, h, sign]() {
10             return applySign(riceWeight(a, h), sign);
11         };
12         auto add_edge = [](Graph& g, int from, int to, double w) {
13             g.addEdge(from, to, w);
14         };
15         return buildByRice<Graph>( numVertices, isDirected,
16                                   a, h, dist,
17                                   gen_weight, add_edge);
18     }

```

Листинг 8: generateRiceGraphByDegrees (Generator.cc)

2.2.4. CollectionUtils: шаблонные утилиты

Статический класс с шаблонными методами для матриц и коллекций.

`makeMatrix<T>(rows, cols, getter)` строит матрицу любого типа, заполняя её вызовами `getter(i, j)`. На его основе `makeSquareMatrix` строит квадратную матрицу по списку вершин — используется в `FileHandler` для сохранения матриц смежности и весов.

`multiplyOptionalMatrix` выполняет матричное умножение над элементами типа `optional<double>`: `nullopt` означает отсутствие пути и не участвует в агрегации. Компаратор передаётся параметром — одна функция работает как для `(min, +)`, так и для `(max, +)` в методе Шимбелла.

```

1 template <typename CompareFunc>
2 static Matrix<std::optional<double>> multiplyOptionalMatrix(
3     Matrix<std::optional<double>> const& a,
4     Matrix<std::optional<double>> const& b,
5     CompareFunc compare)
6 {
7     int size = static_cast<int>(a.size());
8     Matrix<optional<double>> result(
9         size, vector<optional<double>>(size, nullopt)
10    );
11     for (int i = 0; i < size; ++i)
12         for (int j = 0; j < size; ++j)
13             for (int k = 0; k < size; ++k)
14                 if (a[i][k].has_value() && b[k][j].has_value()) {
15                     double d = a[i][k].value() + b[k][j].value();
16                     if (!result[i][j].has_value() ||
17                         compare(d, result[i][j].value()))
18                         result[i][j] = d;
19                 }
20     return result;
21 }

```

Листинг 9: Обобщённое матричное умножение (CollectionUtils.h)

2.2.5. FileHandler: сохранение данных

Статический класс. Все методы принимают имя файла и объект для сохранения. `saveGraph` сохраняет список рёбер в формате `u v weight` по одному ребру на строку — именно этот формат читает `GraphLoader` в Python. `savePaths` пишет число путей первой строкой, затем каждый путь как разделённую последовательность вершин. Приватный шаблонный метод, предназначенный для чтения матрицы `saveSquareMatrix` используется публичными методами `saveAdjacencyMatrix`, `saveWeightMatrix`, `saveCapacityMatrix` и `saveCostMatrix` — все они отличаются только геттером, который передаётся как лямбда.

2.2.6. Visualizer: вызов Python

Статический класс. Каждый метод формирует список строковых аргументов и вызывает нужный скрипт через `runPythonScript`. Вызов идёт через `system()` с построенной командной строкой.

`draw` вызывает `plot_network.py` с флагом `--type graph` или `--type flow`. `drawPaths` — `plot_paths.py` с дополнительным файлом путей. `drawMatrix` — `plot_matrix.py`. `drawColoredGraph` — `plot_colored_graph.py` с файлом раскраски вершин.

`DrawDataConfig` хранит конфигурацию путей и заголовков для каждого пункта меню в `map<int, DrawData>`. Все пути к файлам собраны в одном месте, что упрощает добавление нового пункта меню.

2.3. Модуль lab1

2.3.1. GraphMetrics: эксцентриситеты

Вход: `Graph const&` (конструктор).

Выход: `MetricsResult` — `unordered_map<int, double>` эксцентриситетов, `vector<int>` центра и диаметральных вершин, `double` радиус и диаметр.

Описание: Граф ациклический и невзвешенный с точки зрения метрик: эксцентриситет считается по числу рёбер. Поэтому используется BFS — приватный метод `bfsSteps`, который возвращает расстояния в шагах от источника. Сложность одного обхода $O(V + E)$, итоговая — $O(V(V + E))$.

После обхода `compute` вычисляет эксцентриситет каждой вершины как максимум расстояний по всем остальным. $+\infty$ означает, что вершина недостижима.

```
1 unordered_map<int, double> GraphMetrics::bfsSteps(int src) const
2 {
3     constexpr double INF = numeric_limits<double>::infinity();
4     std::unordered_map<int, double> dist;
5     for (int v : m_graph_.vertexIds()) dist[v] = INF;
6     dist[src] = 0.0;
7
8     std::queue<int> q;
```



```

9      q.push(src);
10     while (!q.empty()) {
11         int u = q.front(); q.pop();
12         for (auto const& [v, w] : m_graph_.neighbors(u)) {
13             if (dist[v] == INF) {
14                 dist[v] = dist[u] + 1.0;
15                 q.push(v);
16             }
17         }
18     }
19     return dist;
20 }
21
22 MetricsResult GraphMetrics::compute() const {
23     constexpr double INF = std::numeric_limits<double>::infinity();
24     auto ids = m_graph_.vertexIds();
25     MetricsResult result;
26     double radius = INF, diameter = 0.0;
27
28     for (int v : ids) {
29         auto dist = bfsSteps(v);
30         double ecc = 0.0;
31         for (int u : ids) {
32             if (u == v) continue;
33             double d = dist.at(u);
34             if (d == INF) { ecc = INF; break; }
35             ecc = std::max(ecc, d);
36         }
37         result.eccentricities[v] = ecc;
38         if (ecc != INF) {
39             radius = std::min(radius, ecc);
40             diameter = std::max(diameter, ecc);
41         }
42     }
43     result.radius = radius == INF ? 0.0 : radius;
44     result.diameter = diameter;
45     for (int v : ids) {
46         double ecc = result.eccentricities[v];
47         if (ecc == result.radius) result.center.push_back(v);
48         if (ecc == result.diameter) result.diametralVerts.push_back(v);
49     }
50     std::sort(result.center.begin(), result.center.end());
51     std::sort(result.diametralVerts.begin(), result.diametralVerts.end());
52     return result;
53 }

```

Листинг 10: BFS и вычисление метрик (GraphMetrics.cc)

2.3.2. ShimbellMethod: матричная степень

Вход: Graph const& (конструктор), int pathLength.

Выход: ShimbellResult — два поля имеют тип вида DistanceMatrix =

vector<vector<optional<double>>>: min_distances и max_distances, и int path_length.

Описание: Начальная матрица строится через вспомогательный метод CollectionUtils::makeSquareMatrix с геттером-лямбдой: диагональ — нули, ребро — его вес из getEdgeWeight, иначе — nullopt. Матрица итеративно умножается $k-1$ раз. Умножение вынесено в приватные методы multiplyMin и multiplyMax, которые делегируют в CollectionUtils::multiplyOptionalMatrix с соответствующим компаратором. Использование nullopt исключает ложные пути через несуществующие рёбра. Краевой случай $k = 0$ возвращает единичную матрицу.

```
1 DistanceMatrix ShimbellMethod::createAdjacencyMatrix() const {
2     return CollectionUtils::makeSquareMatrix<optional<double>>(
3         m_vertex_ids_,
4         [&](int u, int v) -> std::optional<double> {
5             if (u == v) return 0.0;
6             auto w = m_graph_.getEdgeWeight(u, v);
7             return w.has_value() ? w : std::nullopt;
8         }
9     );
10 }
11
12 ShimbellResult const& ShimbellMethod::compute(int pathLength) {
13     if (pathLength == 0) {
14         DistanceMatrix identity(m_size_,
15             vector<optional<double>>(m_size_, nullopt)
16         );
17         for (int i = 0; i < m_size_; ++i) identity[i][i] = 1;
18         result_ = ShimbellResult{identity, identity, 0};
19         return result_;
20     }
21
22     DistanceMatrix current_min = createAdjacencyMatrix();
23     DistanceMatrix current_max = createAdjacencyMatrix();
24
25     DistanceMatrix base_min    = current_min;
26     DistanceMatrix base_max    = current_max;
27
28     for (int step = 2; step <= pathLength; ++step) {
29         current_min = multiplyMin(current_min, base_min);
30         current_max = multiplyMax(current_max, base_max);
31     }
32     result_ = ShimbellResult{
33         .min_distances = current_min,
34         .max_distances = current_max,
35         .path_length   = pathLength
36     };
37     return result_;
38 }
39
40 DistanceMatrix ShimbellMethod::multiplyMin(DistanceMatrix const& a,
41     DistanceMatrix const& b) const {
```

```

41     return CollectionUtils::multiplyOptionalMatrix(
42         a, b, std::less<>()
43     );
44 }
45
46 DistanceMatrix ShimbellMethod::multiplyMax(DistanceMatrix const& a,
47     DistanceMatrix const& b) const {
48     return CollectionUtils::multiplyOptionalMatrix(
49         a, b, std::greater<>()
50     );
51 }

```

Листинг 11: Матрица смежности и метод Шимбелла (ShimbellMethod.cc)

2.3.3. PathCounter: подсчёт маршрутов

Вход: Graph const& (конструктор), int from, int to.

Выход: PathMatrix = vector<vector<int>> — все пути от from до to.

Описание: Публичный метод getAllPaths сначала проверяет наличие обеих вершин через hasVertex и возвращает пустой результат при отсутствии, не запуская обход. Затем вызывается рекурсивный backtrackAllPaths — DFS с маской посещённых вершин unordered_map<int, bool>. При достижении цели текущий путь копируется в результат; при выходе из рекурсии вершина снимается с отметки (*backtracking*). Сложность: $O(n!)$ в худшем случае.

```

1 PathMatrix PathCounter::getAllPaths(int from, int to) {
2     if (!m_graph_.hasVertex(from) || !m_graph_.hasVertex(to))
3         return {};
4     PathMatrix all_paths;
5     std::vector<int> current_path;
6     std::unordered_map<int, bool> visited;
7     backtrackAllPaths(from, to, visited,
8         current_path, all_paths
9     );
10    return all_paths;
11 }
12
13 void PathCounter::backtrackAllPaths(
14     int current, int target,
15     std::unordered_map<int, bool>& visited,
16     std::vector<int>& currentPath, PathMatrix& allPaths)
17 {
18     currentPath.push_back(current);
19     visited[current] = true;
20     if (current == target) {
21         allPaths.push_back(currentPath);
22     } else {
23         for (auto const& [neighborId, _] : m_graph_.neighbors(current))
24             if (!visited[neighborId])
25                 backtrackAllPaths(neighborId, target, visited,
26                     currentPath, allPaths);

```

```

27     }
28     currentPath.pop_back();
29     visited[current] = false;
30 }

```

Листинг 12: Backtracking-поиск маршрутов (PathCounter.cc)

2.4. Модуль lab2

2.4.1. BFS: обход рёбер

Вход: Graph const& (конструктор), int start.

Выход: BFSResult — visited_order (vector<int>), edges_traversed (vector<pair<int, int>>), parent (unordered_map<int, int>), dist_steps (unordered_map<int, int>), iterations (size_t).

Описание: BFS с очередью. Счётчик iterations инкрементируется при каждом рассмотрении ребра (u, v) и его извлечения из очереди. dist_steps[v] = -1 означает недостижимость.

```

1 BFSResult BFS::traverse(int start) const {
2     BFSResult result;
3     for (int v : m_graph_.vertexIds()) {
4         result.dist_steps[v] = -1;
5         result.parent[v] = -1;
6     }
7     result.dist_steps[start] = 0;
8     result.visited_order.push_back(start);
9
10    std::queue<int> q;
11    q.push(start);
12    while (!q.empty()) {
13        int u = q.front(); q.pop();
14        ++result.iterations;
15        for (auto const& [v, w] : m_graph_.neighbors(u)) {
16            ++result.iterations;
17            if (result.dist_steps[v] == -1) {
18                result.dist_steps[v] = result.dist_steps[u] + 1;
19                result.parent[v] = u;
20                result.edges_traversed.emplace_back(u, v);
21                result.visited_order.push_back(v);
22                q.push(v);
23            }
24        }
25    }
26    return result;
27 }

```

Листинг 13: Обход рёбер в ширину (BFS.cc)

2.4.2. Dijkstra: кратчайшие пути

Вход: Graph const& (конструктор), int start.

Выход: DijkstraResult — dist (unordered_map<int, double>), parent (unordered_map<int, int>), iterations (size_t).

Описание: Min-heap на priority_queue с std::greater<>. Счётчик iterations инкрементируется при каждой попытке релаксации ребра и работой с кучей. Устаревшие записи в куче отфильтровываются проверкой $d > \text{dist}[u]$. Путь восстанавливается через PathUtils::reconstructPath по массиву parent. Сложность: $O((V + E) \log V)$.

```
1 DijkstraResult Dijkstra::compute(int start) const {
2     DijkstraResult result;
3     for (int v : m_graph_.vertexIds()) {
4         result.dist[v] = INF;
5         result.parent[v] = -1;
6     }
7     result.dist[start] = 0.0;
8
9     using Entry = std::pair<double, int>;
10    std::priority_queue<Entry, std::vector<Entry>,
11                        std::greater<Entry>> pq;
12    pq.push({0.0, start});
13
14    while (!pq.empty()) {
15        auto [d, u] = pq.top(); pq.pop();
16        ++result.iterations;
17        if (d > result.dist[u]) continue;
18        for (auto const& [v, w] : m_graph_.neighbors(u)) {
19            ++result.iterations;
20            double nd = result.dist[u] + w;
21            if (nd < result.dist[v]) {
22                result.dist[v] = nd;
23                result.parent[v] = u;
24                pq.push({nd, v});
25            }
26        }
27    }
28    return result;
29 }
```

Листинг 14: Алгоритм Дейкстры (Dijkstra.cc)

2.4.3. Comparator: сравнение алгоритмов

Вход: Graph const&, int start.

Выход: CompareResults = vector<CompareResult>, каждый элемент содержит algorithm_name и iterations.

Описание: Запускает BFS и Дейкстру на одном графе с одной стартовой вершиной и собирает счётчики итераций в единый вектор.

```

1 Comparator::CompareResults
2 Comparator::compare(Graph const& graph, int start) {
3     CompareResults results;
4     {
5         BFS bfs(graph);
6         auto r = bfs.traverse(start);
7         results.push_back({"BFS", r.iterations});
8     }
9     {
10        Dijkstra dijk(graph);
11        auto r = dijk.compute(start);
12        results.push_back({"Dijkstra", r.iterations});
13    }
14    return results;
15 }

```

Листинг 15: Сравнение алгоритмов (Comparator.cc)

2.5. Модуль lab3

2.5.1. FlowNetwork: сеть потоков

Вход: флаг ориентированности (конструктор, по умолчанию true).

Выход: `unique_ptr<FlowNetwork>`.

Описание: `FlowNetwork` наследует `GraphBase<FlowVertex, FlowEdge>`. Структура `FlowEdge` хранит `from`, `to`, `capacity`, `cost` и `flow`. При добавлении ребра через `addEdge` автоматически создаётся обратное ребро с нулевой пропускной способностью и отрицательной стоимостью — это стандартное требование алгоритмов на остаточных сетях. Остаточная пропускная способность вычисляется как $res(u, v) = capacity(u, v) - flow(u, v)$. Метод `addFlow` обновляет поток симметрично: увеличивает прямое ребро и уменьшает обратное.

```

1 bool FlowNetwork::addEdge(int from, int to,
2                           double capacity, double cost)
3 {
4     FlowEdge forward_edge(from, to, capacity, cost, 0.0);
5     FlowEdge backward_edge(to, from, 0.0, -cost, 0.0);
6
7     bool added = GraphBase::addEdge(from, to, forward_edge);
8     if (added) {
9         GraphBase::addEdge(to, from, backward_edge);
10        m_vertices_[to]->addNeighbor(from);
11    }
12    return added;
13 }
14
15 void FlowNetwork::addFlow(int from, int to, double flow) {
16     auto* edge_forward = getEdgeMutable(from, to);
17     auto* edge_backward = getEdgeMutable(to, from);
18     if (edge_forward) edge_forward->flow += flow;

```

```

19     if (edge_backward) edge_backward->flow -= flow;
20 }

```

Листинг 16: Добавление ребра и обратного ребра (FlowNetwork.cc)

2.5.2. MaxFlow: алгоритм Форда–Фалкерсона

Вход: FlowNetwork& (конструктор), int source, int sink, bool enableLogging.

Выход: double — значение максимального потока. Побочный эффект — вектор m_snapshots_ для анимации.

Описание: Реализован метод Форда–Фалкерсона с BFS для поиска увеличивающего пути. На каждой итерации BFS ищет путь в остаточной сети — ребро рассматривается только если $res(u, v) > 0$. Пропускная способность пути — минимум остаточных пропускных способностей по всем рёбрам — вычисляется через PathUtils::getMinPathValue. Поток увеличивается через PathUtils::forEachEdgeInPath. При включённом логировании после каждой итерации фиксируется снимок состояния сети FlowSnapshot — он используется для построения GIF-анимации роста потока. Сложность: $O(V \cdot E^2)$.

```

1 double MaxFlow::fordFulkerson(int source, int sink,
2                               bool enableLogging)
3 {
4     double max_flow = 0.0;
5     std::unordered_map<int, int> parent;
6     int step = 0;
7
8     resetFlows(m_network_);
9     m_snapshots_.clear();
10    if (enableLogging) captureSnapshot(step++, max_flow);
11
12    while (bfs(source, sink, parent)) {
13        auto calc_residual = [this](int u, int v) {
14            return m_network_.getResidualCapacity(u, v);
15        };
16        double path_flow = PathUtils<double>::getMinPathValue(
17            source, sink, parent, calc_residual);
18
19        PathUtils<double>::forEachEdgeInPath(
20            source, sink, parent,
21            [this, path_flow](int u, int v) {
22                m_network_.addFlow(u, v, path_flow);
23            });
24
25        max_flow += path_flow;
26        if (enableLogging) captureSnapshot(step++, max_flow);
27    }
28    return max_flow;
29 }

```

Листинг 17: Алгоритм Форда–Фалкерсона (MaxFlow.cc)

2.5.3. MinCostFlow: поток минимальной стоимости

Вход: FlowNetwork& (конструктор), int source, int sink, double targetFlow.

Выход: Result — структура с полями flow, cost, success и path последнего найденного пути.

Описание: Алгоритм последовательно ищет кратчайший путь в остаточной сети по стоимости через алгоритм Беллмана–Форда и направляет по нему максимально возможный поток, не превышающий $targetFlow - currentFlow$. Беллман–Форд корректно обрабатывает отрицательные стоимости обратных рёбер. По условию задания целевой поток берётся как $\lfloor 2/3 \cdot maxFlow \rfloor$. Сложность одной итерации: $O(V \cdot E)$.

```
1 MinCostFlow::Result MinCostFlow::findMinCostFlow(  
2     int source, int sink, int targetFlow)  
3 {  
4     Result result;  
5     resetFlows(m_network_);  
6     double current_flow = 0.0, total_cost = 0.0;  
7     std::unordered_map<int, double> dist;  
8     std::unordered_map<int, int> parent;  
9  
10    while (current_flow < targetFlow  
11           && bellmanFord(source, sink, dist, parent))  
12    {  
13        auto calc_residual = [this](int u, int v) {  
14            return m_network_.getResidualCapacity(u, v);  
15        };  
16        double path_flow = std::min(  
17            targetFlow - current_flow,  
18            PathUtils<double>::getMinPathValue(  
19                source, sink, parent, calc_residual));  
20  
21        result.path = PathUtils<double>::reconstructPath(  
22            source, sink, parent);  
23  
24        PathUtils<double>::forEachEdgeInPath(  
25            source, sink, parent,  
26            [this, &total_cost, path_flow](int u, int v) {  
27                m_network_.addFlow(u, v, path_flow);  
28                total_cost += path_flow  
29                    * m_network_.getCost(u, v);  
30            });  
31  
32        current_flow += path_flow;  
33    }  
34    result.flow = current_flow;  
35    result.cost = total_cost;  
36    result.success = (current_flow == targetFlow);  
37    return result;  
38 }
```

Листинг 18: Поток минимальной стоимости (MinCostFlow.cc)

2.5.4. PathUtils: вспомогательные операции на путях

PathUtils<T> — шаблонный статический класс для работы с путями, заданными массивом предшественников unordered_map<int, int> parent. Используется как в MaxFlow, так и в MinCostFlow, а также в Dijkstra из лабораторной 2.

getMinPathValue вычисляет минимальное значение геттера по всем рёбрам пути — используется для нахождения пропускной способности пути. forEachEdgeT применяет произвольный функтор к каждому ребру пути — использует

2.6. Модуль lab4

2.6.1. KirchhoffTheorem: подсчёт числа остовных деревьев

Вход: Graph const& — неориентированный граф.

Выход: size_t — число различных остовных деревьев.

Описание: Метод countSpanningTrees строит матрицу Лапласа L размера $n \times n$ по формуле: $\ell_{ii} = \deg v_i$ на диагонали, $\ell_{ij} = -1$ если ребро (i, j) существует, иначе $\ell_{ij} = 0$. Матрица строится через вспомогательный шаблон CollectionUtils::makeMatrix, принимающий лямбду-заполнитель. Затем удаляется нулевая строка и нулевой столбец — получается минор L_{00} размера $(n - 1) \times (n - 1)$, определитель которого по теореме Кирхгофа равен числу остовных деревьев. Определитель вычисляется рекурсивным разложением по первой строке с построением подматриц: для каждого элемента $L_{00}[0][p]$ формируется подматрица размера $(n - 2) \times (n - 2)$ вычеркиванием нулевой строки и столбца p , после чего рекурсивно вычисляется её определитель. Базовые случаи — матрицы 1×1 и 2×2 вычисляются напрямую. Результат округляется до ближайшего целого через std::llround и зажимается снизу нулём через std::max, чтобы исключить артефакты плавающей точки. Граничные случаи: граф из нуля вершин возвращает 0, из одной вершины — 1. Сложность: $O(n^3)$, пространственная: $O(n^2)$.

```
1 size_t KirchhoffTheorem::countSpanningTrees(Graph const& graph);
2
3 Matrix laplacian = CollectionUtils::makeMatrix<double>(
4     graph.vertexIds(), graph.vertexIds(),
5     [&graph](int i, int j) {
6         if (i == j)
7             return static_cast<double>(graph.degree(i));
8         return graph.hasEdge(i, j) ? -1.0 : 0.0;
9     });
10
11 Matrix minor(n - 1, std::vector<double>(n - 1));
12 for (size_t i = 1; i < n; ++i)
13     for (size_t j = 1; j < n; ++j)
14         minor[i - 1][j - 1] = laplacian[i][j];
15
16 return static_cast<size_t>(
```

```

17     std::llround(std::max(0.0, determinant(minor))));
18
19 double KirchhoffTheorem::determinant(Matrix const& matrix);
20
21 double det = 0.0;
22 for (size_t p = 0; p < n; ++p) {
23     Matrix submatrix(n - 1, std::vector<double>(n - 1));
24     for (size_t i = 1; i < n; ++i) {
25         size_t col = 0;
26         for (size_t j = 0; j < n; ++j)
27             if (j != p)
28                 submatrix[i - 1][col++] = matrix[i][j];
29     }
30     double cofactor = (p % 2 == 0 ? 1 : -1) * matrix[0][p];
31     det += cofactor * determinant(submatrix);
32 }

```

Листинг 19: Построение матрицы Лапласа, минора и вычисление определителя (KirchhoffTheorem.cc)

2.6.2. Boruvka: алгоритм Борувки

Вход: `Graph const&` — связный неориентированный взвешенный граф.

Выход: `unique_ptr<Graph>` — минимальное остовное дерево; `nullptr` если граф ориентирован, пуст или несвязен.

Описание: Метод `buildMST` реализует алгоритм Борувки. На этапе инициализации список всех рёбер собирается однократно — каждое ребро добавляется ровно один раз благодаря условию $u < v$, исключающему дублирование для неориентированного графа. Все вершины добавляются в результирующий граф `mst` заранее.

Компоненты связности отслеживаются внутренней структурой `UnionFind`. Метод `find` реализует сжатие пути: при каждом вызове родитель вершины перезаписывается непосредственно на корень компоненты. Метод `unite` объединяет две компоненты по рангу: корень с меньшим рангом подвешивается под корень с большим; при равных рангах ранг нового корня увеличивается на 1.

На каждой итерации главного цикла для каждого ребра (u, v) определяются корни компонент `set_u` и `set_v`; ребро пропускается если они совпадают. Иначе для каждой из двух компонент обновляется словарь `cheapest`: кандидат заменяется только если его вес строго меньше текущего минимума. После обхода всех рёбер каждое ребро из `cheapest` перед добавлением в остов проверяется повторно — условие `find(u) == find(v)` отсеивает дубли, возникающие когда одно ребро является наилучшим сразу для двух компонент. Принятое ребро добавляется в `mst`, компоненты объединяются, счётчик `components` уменьшается. Итерации продолжаются пока `components > 1`. Если за итерацию ни одного слияния не произошло — граф несвязен, возвращается `nullptr`. Сложность: $O(E \log V)$, пространственная: $O(V + E)$.

```

1 std::unique_ptr<Graph> Boruvka::buildMST(Graph const& graph);
2 int Boruvka::UnionFind::find(int v);
3 void Boruvka::UnionFind::unite(int u, int v);
4
5 std::vector<Edge> edges;
6 for (int u : vertices)
7     for (auto const& [v, weight] : graph.neighbors(u))
8         if (u < v)
9             edges.push_back({u, v, weight});
10
11 while (components > 1) {
12     std::unordered_map<int, Edge> cheapest;
13
14     for (auto const& edge : edges) {
15         int set_u = uf.find(edge.from);
16         int set_v = uf.find(edge.to);
17         if (set_u == set_v) continue;
18
19         auto update_cheapest = [&cheapest](int component,
20                                           Edge const& candidate) {
21             auto it = cheapest.find(component);
22             if (it == cheapest.end()
23                 || candidate.weight < it->second.weight)
24                 cheapest[component] = candidate;
25         };
26         update_cheapest(set_u, edge);
27         update_cheapest(set_v, edge);
28     }
29
30     if (cheapest.empty()) return nullptr;
31
32     bool merged_any = false;
33     for (auto const& [component, edge] : cheapest) {
34         (void)component;
35         if (uf.find(edge.from) == uf.find(edge.to)) continue;
36         uf.unite(edge.from, edge.to);
37         mst->addEdge(edge.from, edge.to, edge.weight);
38         --components;
39         merged_any = true;
40     }
41     if (!merged_any) return nullptr;
42 }

```

Листинг 20: Сбор рёбер, выбор cheapestого ребра для каждой компоненты и слияние (Boruvka.cc)

2.6.3. PruferCode: кодирование и декодирование кода Прюфера

Вход (encode): Graph const& — дерево (как правило, результат алгоритма Борувки).

Выход (encode): PruferEncoding — структура с полями: sequence — по-

следовательность Прюфера длины $n - 2$; `vertices` — отсортированный список вершин; `edgeWeights` — словарь весов рёбер, индексированный парой $(\min(u, v), \max(u, v))$.

Описание (encode): Перед началом кодирования все веса рёбер сохраняются в `edgeWeights` — это необходимо для последующего восстановления взвешенного дерева при декодировании, поскольку в ходе кодирования структура дерева логически разрушается. Степени вершин хранятся в локальном словаре `degree`; граф не модифицируется. На каждом из $n - 2$ шагов линейным поиском по отсортированному списку вершин находится висющаяся вершина с минимальным номером (степень равна 1). Среди её соседей выбирается тот, чья степень строго больше 0 — он записывается в `sequence`. Затем степени обоих концов уменьшаются на 1. Для деревьев из ≤ 2 вершин последовательность остаётся пустой — граничный случай обрабатывается явным ранним выходом.

Вход (decode): `PruferEncoding const&` — структура, полученная при кодировании.

Выход (decode): `unique_ptr<Graph>` — восстановленное взвешенное дерево.

Описание (decode): Степени вершин восстанавливаются из кода: каждая вершина получает начальную степень 1, затем степень увеличивается на 1 за каждое вхождение в `sequence`. На каждом шаге линейным поиском по отсортированному списку вершин находится наименьшая вершина со степенью 1 — она не встречается в ещё не обработанной части кода, поскольку её степень не будет увеличиваться. Добавляется ребро между найденной вершиной и текущим элементом `sequence`; вес берётся из `edgeWeights` по ключу $(\min, \max)$, при отсутствии — подставляется 1.0. Степени обоих концов уменьшаются на 1. После $n - 2$ шагов ровно две вершины имеют степень 1 — они соединяются последним ребром. Сложность кодирования и декодирования: $O(n^2)$, пространственная: $O(n)$.

```
1 PruferCode::PruferEncoding PruferCode::encode(Graph const& tree);
2
3 for (int u : vertices)
4     for (auto const& [v, weight] : tree.neighbors(u))
5         if (u < v)
6             result.edgeWeights[{u, v}] = weight;
7
8 for (size_t i = 0; i < vertices.size() - 2; ++i) {
9     int leaf = -1;
10    for (int v : vertices)
11        if (degree[v] == 1) { leaf = v; break; }
12
13    for (auto const& [u, weight] : tree.neighbors(leaf)) {
14        if (degree[u] > 0) {
15            result.sequence.push_back(u);
16            degree[leaf]--;
17            degree[u]--;
18            break;
19        }
20    }
```

21 }

Листинг 21: Кодирование: сохранение весов, поиск листа и заполнение последовательности (PruferCode.cc)

```

1 std::unique_ptr<Graph> PruferCode::decode(
2     PruferEncoding const& encoding);
3
4 std::unordered_map<int, int> degree;
5 for (int v : vertices) degree[v] = 1;
6 for (int v : encoding.sequence) degree[v]++;
7
8 for (int v : encoding.sequence) {
9     int leaf = -1;
10    for (int candidate : vertices)
11        if (degree[candidate] == 1) { leaf = candidate; break; }
12
13    tree->addEdge(leaf, v, lookupWeight(leaf, v));
14    degree[leaf]--;
15    degree[v]--;
16 }
17
18 int v1 = -1, v2 = -1;
19 for (int candidate : vertices)
20     if (degree[candidate] == 1) {
21         if (v1 == -1) v1 = candidate;
22         else { v2 = candidate; break; }
23     }
24 if (v1 != -1 && v2 != -1)
25     tree->addEdge(v1, v2, lookupWeight(v1, v2));

```

Листинг 22: Декодирование: восстановление степеней, добавление рёбер и последнее ребро (PruferCode.cc)

2.6.4. GraphCombinatorics: минимальная раскраска графа

Вход: Graph const& — произвольный граф (исходный или МОД — по выбору пользователя).

Выход: vector<int> — вектор цветов длины n , где элемент с индексом i соответствует i -й вершине в отсортированном порядке; цвета нумеруются с 0.

Описание: Метод findMinColoring перебирает число цветов k от 1 до n включительно. Для каждого k вектор colors сбрасывается в -1 и запускается рекурсивный backtrackColoring. На каждом шаге рекурсии обрабатывается вершина с индексом index в отсортированном порядке. Для неё перебираются цвета от 0 до $k - 1$: цвет считается допустимым, если ни один из уже окрашенных соседей не имеет такого же цвета — соседи ищутся через graph.getNeighbors, их индекс в векторе цветов определяется через vertex_to_index. Если допустимый цвет найден, он присваивается и рекурсия идёт на следующую вершину; при неудаче цвет сбрасывается в -1 и перебирается следующий.

Первое k , при котором рекурсия вернула `true`, является хроматическим числом $\chi(G)$ — найденная раскраска сохраняется в `best_colors` и цикл прерывается. После завершения правильность раскраски дополнительно верифицируется методом `isValidColoring`, обходящим все рёбра графа. Сложность в худшем случае: $O(n \cdot k^n)$, пространственная: $O(n)$.

```

1 VerticesSet GraphCombinatorics::findMinColoring(
2     Graph const& graph);
3
4 bool GraphCombinatorics::backtrackColoring(
5     Graph const& graph, int index,
6     VerticesSet const& vertices, VerticesSet& colors,
7     int num_colors,
8     std::unordered_map<int, int> const& vertex_to_index);
9
10 bool GraphCombinatorics::isValidColoring(
11     Graph const& graph, VerticesSet const& colors,
12     VerticesSet const& vertices,
13     std::unordered_map<int, int> const& vertex_to_index) const;
14
15 for (int num_colors = 1; num_colors <= static_cast<int>(n); ++num_colors) {
16     std::fill(colors.begin(), colors.end(), -1);
17     if (backtrackColoring(graph, 0, vertices, colors,
18         num_colors, vertex_to_index)) {
19         best_colors = colors;
20         break;
21     }
22 }
23
24 int vertex = vertices[index];
25 for (int color = 0; color < num_colors; ++color) {
26     bool canColor = true;
27     for (int neighbor : graph.getNeighbors(vertex)) {
28         int neighbor_idx = vertex_to_index.at(neighbor);
29         if (colors[neighbor_idx] == color) {
30             canColor = false;
31             break;
32         }
33     }
34     if (canColor) {
35         colors[index] = color;
36         if (backtrackColoring(graph, index + 1, vertices, colors,
37             num_colors, vertex_to_index))
38             return true;
39         colors[index] = -1;
40     }
41 }

```

Листинг 23: Перебор числа цветов, backtracking и проверка допустимости (GraphCombinatorics.cc)

2.7. Модуль lab5

2.7.1. EulerianCycle: проверка и нахождение эйлерова цикла

Вход (конструктор): Graph& — неориентированный граф.

Выход (findCycle): std::optional<vector<int>> — последовательность вершин эйлерова цикла или пути; std::nullopt если граф не является ни эйлеровым, ни полуэйлеровым.

Описание: Метод isEulerian проверяет отсутствие вершин нечётной степени и связность графа; isSemiEulerian — ровно две вершины нечётной степени при том же условии связности. Связность определяется BFS-обходом по ненулевым вершинам через hasSingleNonZeroComponent.

Метод makeEulerian выполняет достройку: сначала соединяет несвязные компоненты цепочкой рёбер, затем итеративно устраняет вершины нечётной степени попарным добавлением рёбер. При выборе пар предпочтение отдаётся вершинам, между которыми ещё нет ребра. Все добавленные рёбра фиксируются в m_added_edges_.

Метод hierholzer реализует алгоритм Иерхольцера итеративно на стеке с вектором usedEdge и указателями ptr[u] для пропуска использованных рёбер без повторного перебора. Сложность: $O(V + E)$, пространственная: $O(V + E)$.

```
1 bool EulerianCycle::isEulerian() const;
2 bool EulerianCycle::isSemiEulerian() const;
3 void EulerianCycle::makeEulerian();
4 std::optional<std::vector<int>> EulerianCycle::hierholzer(
5     int start, int totalEdges) const;
6
7 stk.push(id2idx[start]);
8 while (!stk.empty()) {
9     int u = stk.top();
10    bool found = false;
11    while (ptr[u] < static_cast<int>(g[u].size())) {
12        auto [v, eid] = g[u][ptr[u]++];
13        if (usedEdge[eid]) continue;
14        usedEdge[eid] = true;
15        stk.push(v);
16        found = true;
17        break;
18    }
19    if (!found) {
20        circuit.push_back(vertices[u]);
21        stk.pop();
22    }
23 }
24 std::reverse(circuit.begin(), circuit.end());
```

Листинг 24: Достройка и алгоритм Иерхольцера (EulerianCycle.cc)

2.7.2. CutSystem: фундаментальная система разрезов

Вход (конструктор): `Graph const& originalGraph, Graph const& mst` — исходный граф и его минимальный остов.

Выход (`buildFundamentalCuts`): `vector<FundamentalCut>` — вектор из $|V|-1$ структур с полями `treeEdge`, `leftComponent`, `rightComponent` и `cutEdges`.

Описание: Для каждого ребра $e \in T$ метод `componentWithoutTreeEdge` BFS-обходом по остову без ребра e находит компоненту A ; дополнение $B = V \setminus A$ строится перебором всех вершин. Рёбра исходного графа, пересекающие границу $A \mid B$, собираются в `cutEdges`, нормализуются и дедуплицируются. Сложность: $O(|V| \cdot (|V| + |E|))$.

Метод `symmetricDifference` кодирует каждое ребро как `uint64` и поддерживает множество `active` по принципу XOR: при повторном вхождении код удаляется, при первом — добавляется. Сложность: $O(k \cdot |\text{Cut}|)$.

```
1 std::vector<CutSystem::FundamentalCut>
2     CutSystem::buildFundamentalCuts() const;
3 std::vector<std::pair<int, int>> CutSystem::symmetricDifference(
4     std::vector<FundamentalCut> const& cuts,
5     std::vector<int> const& selectedIndices) const;
6
7 for (auto const& edge : mst_.edges()) {
8     auto treeEdge = normalizeEdge(edge.from, edge.to);
9     auto left = componentWithoutTreeEdge(treeEdge.first, treeEdge);
10    for (auto const& e : originalGraph_.edges()) {
11        auto ge = normalizeEdge(e.from, e.to);
12        if (left.contains(ge.first) != left.contains(ge.second))
13            cut.cutEdges.push_back(ge);
14    }
15 }
16
17 for (int idx : selectedIndices)
18     for (auto const& edge : cuts[idx].cutEdges) {
19         auto code = encode(edge);
20         if (active.contains(code)) active.erase(code);
21         else active.insert(code);
22     }
```

Листинг 25: Построение ФСР и симметрическая разность (CutSystem.cc)

2.8. Визуализация: Python-скрипты

Скрипты находятся в `scripts/` и делятся на ядро (`core/`) и скрипты отрисовки (`visualization/`). Ядро содержит три компонента: `config.py` определяет все параметры отображения: цвета, размеры, колормапы. Неизменяемые датаклассы гарантируют, что скрипт случайно не перезапишет глобальные настройки. `GraphLoader` читает текстовые файлы в граф `networkx`. `Renderer` — обёртка над `matplotlib/networkx` с методами `setup_plot`,

`compute_layout`, `draw_nodes`, `draw_edges`, `draw_labels`, `finalize`. Это убивает дублирование конфигурации в каждом скрипте.

Каждый скрипт в `visualization/` принимает аргументы через `argparse`, читает данные через `GraphLoader`, рисует через `Renderer` и сохраняет в файл. `plot_network.py` обрабатывает оба типа — граф и сеть потоков — через флаг `--type`. Толщина рёбра для графа пропорциональна весу, для потока — текущему потоку; цвет ребра потока задаётся через колормап по коэффициенту загрузки *flow/capacity*. `plot_paths.py` получает файл с путями, строит словарь рёбер, входящих в пути, и выделяет их цветом: каждый путь своим цветом из `cm.rainbow`. `plot_matrix.py` рисует тепловую карту через `seaborn` с аннотациями значений.

2.9. Веб-интерфейс

Веб-интерфейс реализован на Streamlit (`web/`). Приложение (`app.py`) запускает исполняемый файл C++ как дочерний процесс через `subprocess.Popen` с открытым `stdin`. Взаимодействие с C++ происходит через `send_command` — метод просто пишет строку в `proc.stdin`. C++ читает её как ввод пользователя в консольном цикле и выполняет нужное действие.

Конфигурация описана в `config.py` через датаклассы. `LabConfig` описывает лабораторную, `ActionConfig` — одно действие внутри неё: номер команды (`execute_cmd`), список параметров ввода (`params`) и список файлов для отображения (`images`). Класс `GraphLabApp` в `ui.py` обходит эту конфигурацию и рендерит виджеты Streamlit динамически.

Дополнительно для лабораторной 1 строится гистограмма весов с наложенной теоретической PDF распределения Рэлея–Райса. PDF вычисляется напрямую в Python через `scipy.special.i0` (функция Бесселя) по той же формуле Вадзинского, что и в C++. Это позволяет визуально сравнить реальное распределение сгенерированных весов с теоретическим.

3. Результаты работы программы

В данном разделе представлены результаты выполнения всех пунктов заданий лабораторных работ 1–5. Для каждого пункта приведён краткий обзор вывода программного комплекса и соответствующая визуализация.

3.1. Лабораторная работа 1

3.1.1. Генерация графа по распределению Рэлея–Райса

Генерируется связный ациклический неориентированный граф. Пользователь вводит число вершин n и параметры распределения a и h . Число рёбер и их распределение по степеням вершин определяются автоматически через softmax-нормировку выборок из распределения Рэлея–Райса. Веса рёбер генерируются по тому же закону.

Визуальное подтверждение корректности генерации весов представлено на рис. 3: гистограмма сгенерированных значений с наложенной теоретической плотностью вероятности $f(z; a, h)$.

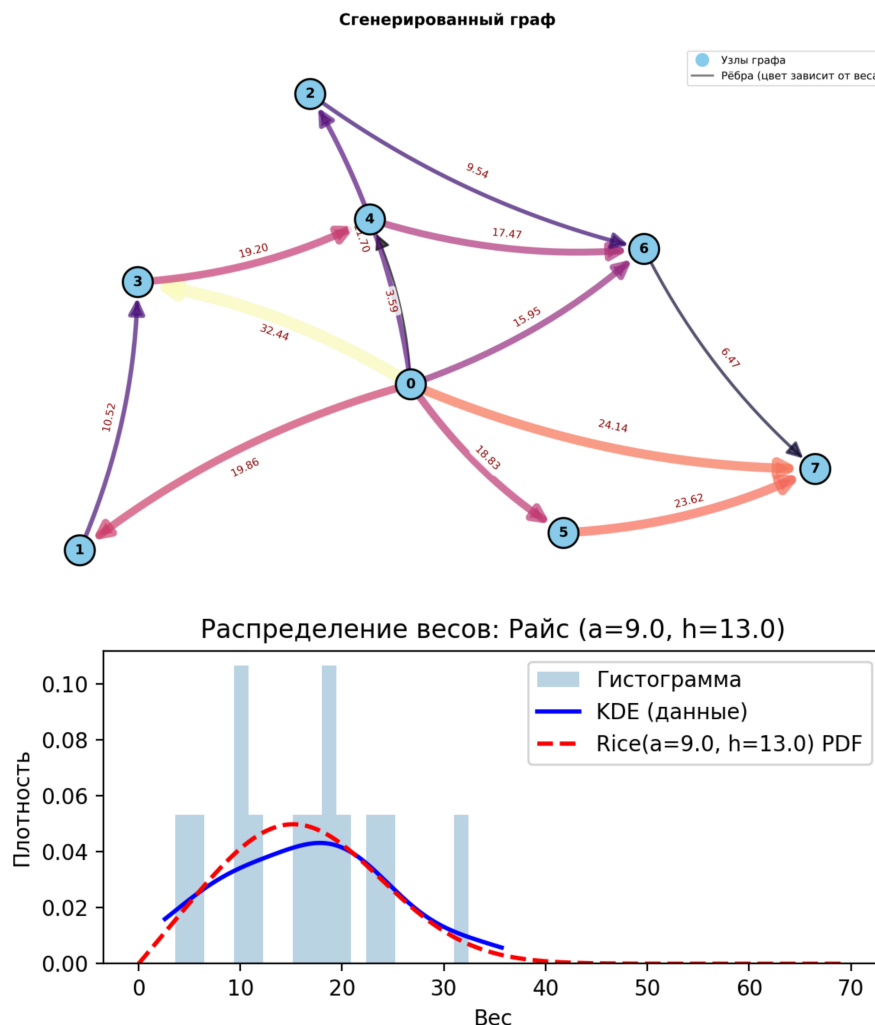


Рис. 3: Сгенерированный граф и распределение весов рёбер

3.1.2. Матрицы смежности и весов

Программа экспортирует матрицу смежности и матрицу весов. Результат приведён на рис. 4.

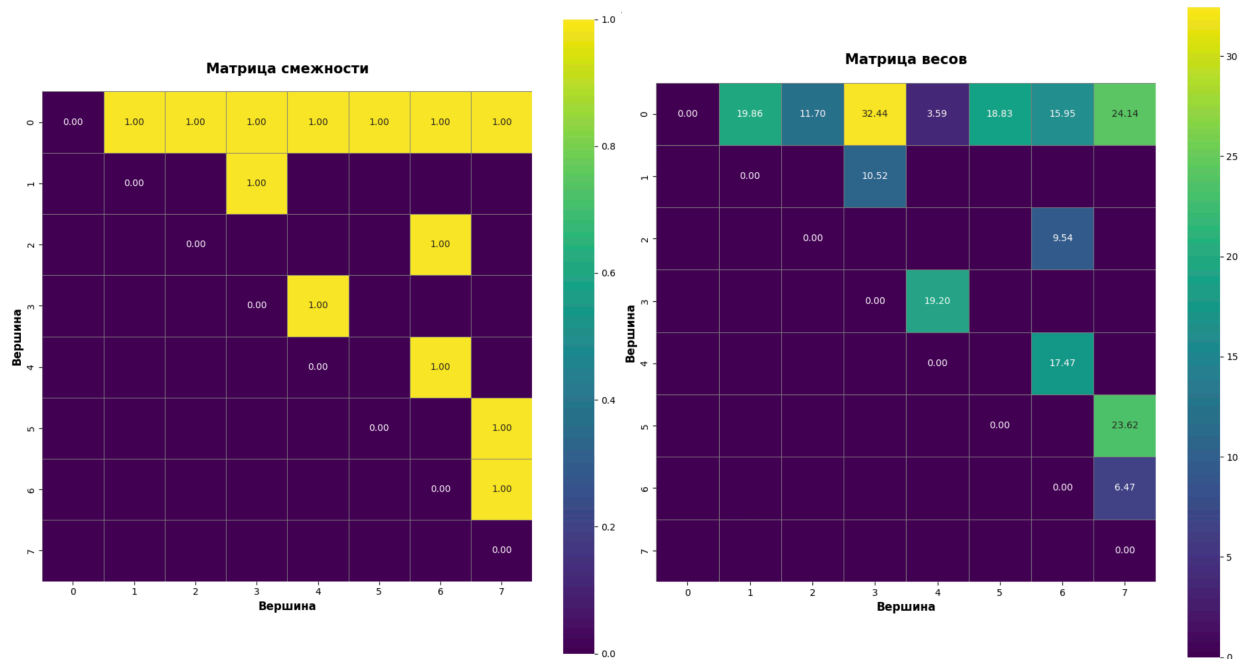


Рис. 4: Матрица смежности (слева) и матрица весов (справа)

3.1.3. Метод Шимбелла

Программа строит две матрицы: минимальных и максимальных суммарных весов маршрутов длины ровно k . Матрицы для $k = V - 1$ показаны на рис. 5.

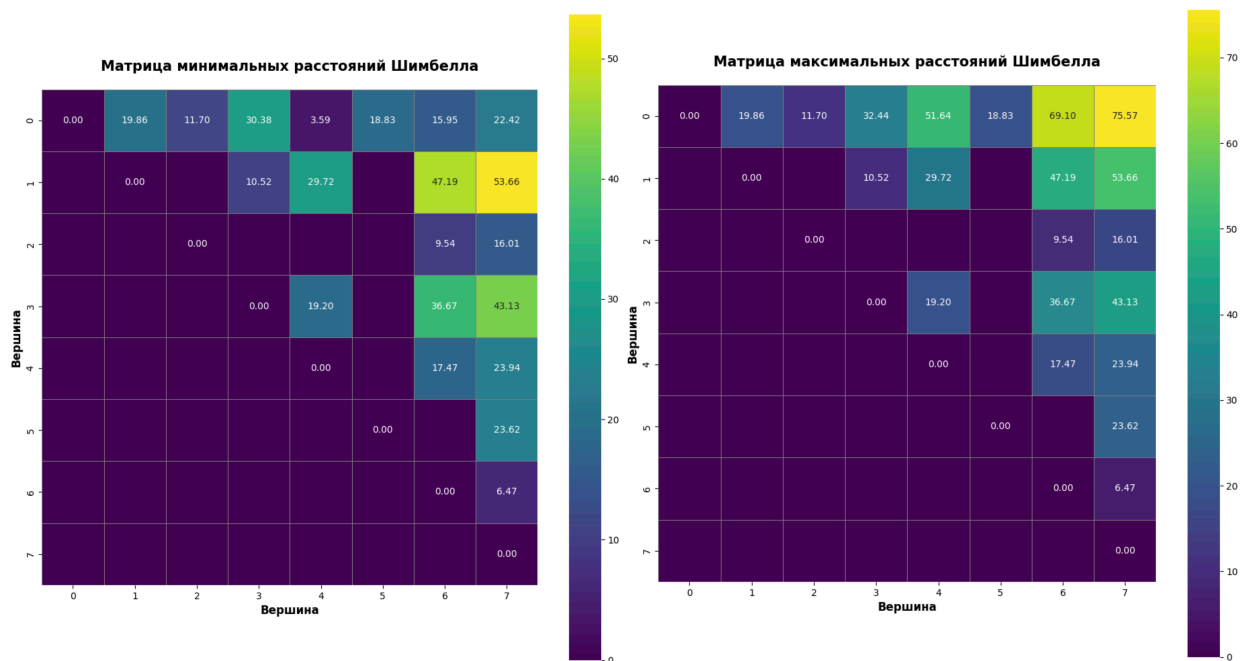


Рис. 5: Матрицы Шимбелла для $k = V - 1$: мин (слева) и макс (справа) пути

3.1.4. Подсчёт всех маршрутов

Пользователь вводит вершину-источник и вершину-цель. Программа находит все простые пути методом поиска с возвратом и выводит их список в консоль. На визуализации (рис. 6) каждый путь выделен своим цветом, рёбра остального графа отображаются серым.

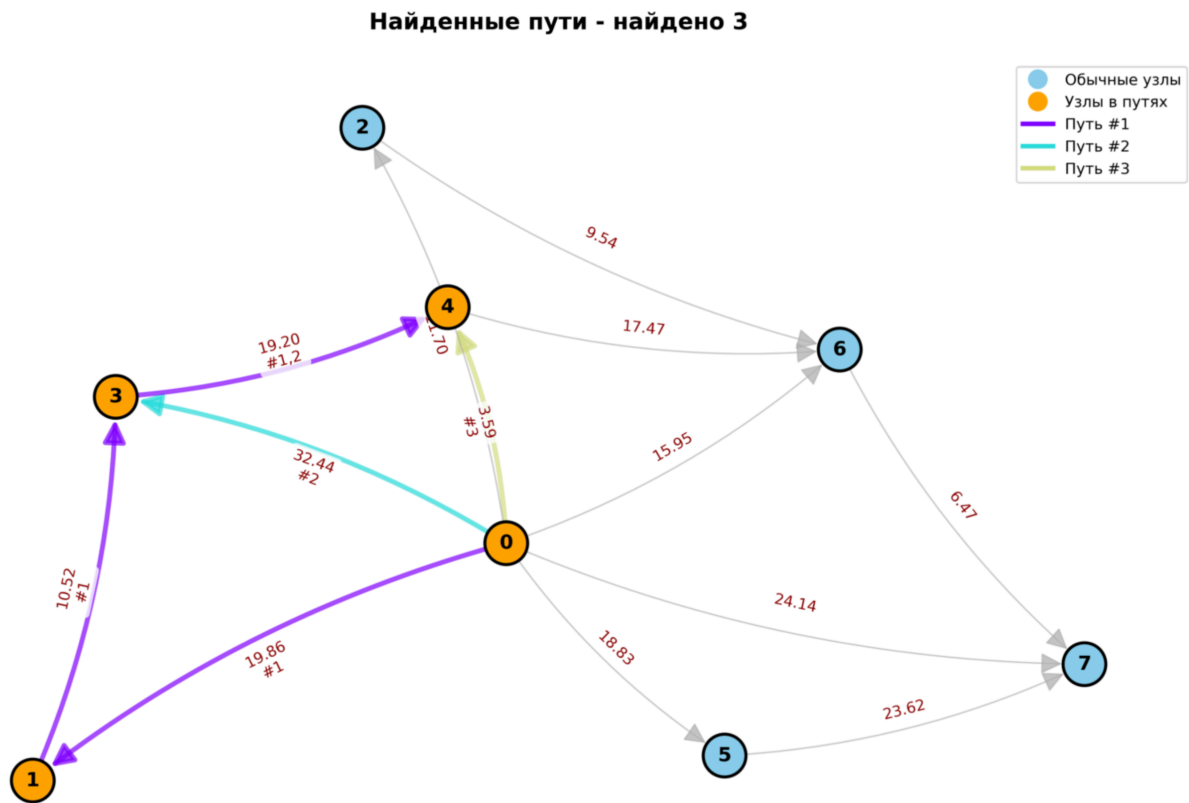


Рис. 6: Все простые маршруты между заданными вершинами

3.1.5. Метрики графа: эксцентриситеты, центр, диаметр

Программа вычисляет эксцентриситет каждой вершины через BFS, находит радиус, диаметр, центральные и диаметральные вершины. На визуализациях (рис. 7, 8) вершины раскрашены по роли: центральные — одним цветом, диаметральные — другим. Текстовый вывод приведён на рис. 9.

Эксцентриситеты, центр и диаметральные вершины

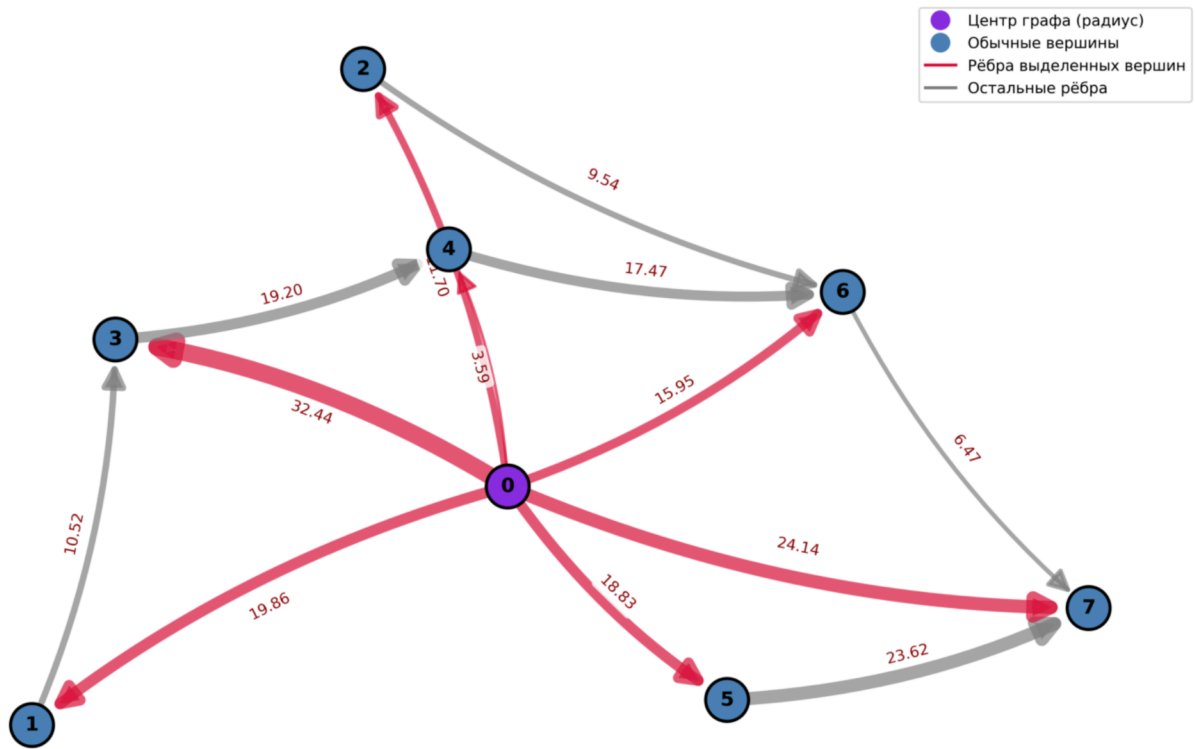


Рис. 7: Орграф с выделением центра

Эксцентриситеты, центр и диаметральные вершины

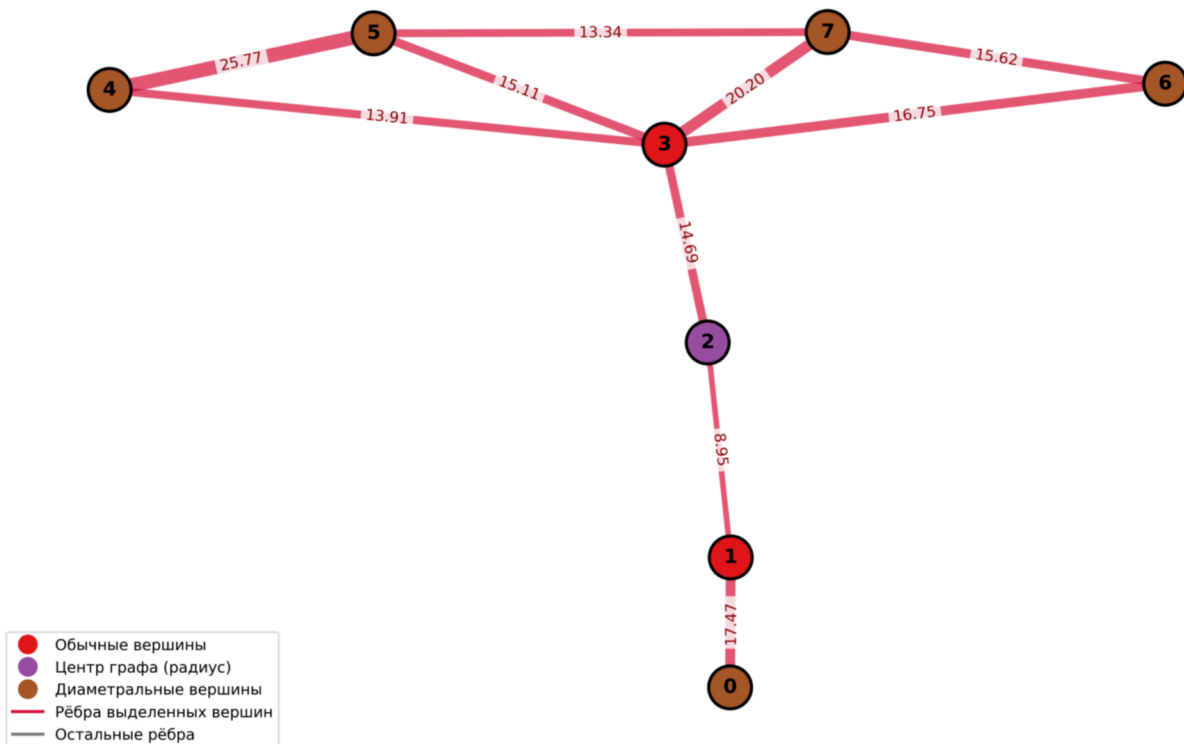


Рис. 8: Граф с выделением центра и диаметральных вершин

Эксцентриситеты вершин:		Эксцентриситеты вершин:	
v0 :	1.000000	v0 :	4.000000
v1 :	+inf	v1 :	3.000000
v2 :	+inf	v2 :	2.000000
v3 :	+inf	v3 :	3.000000
v4 :	+inf	v4 :	4.000000
v5 :	+inf	v5 :	4.000000
v6 :	+inf	v6 :	4.000000
v7 :	+inf	v7 :	4.000000
Радиус графа :	1	Радиус графа :	2
Диаметр графа :	1	Диаметр графа :	4

Рис. 9: Текстовый вывод метрик графа

В орграфе только вершина v_0 имеет конечный эксцентриситет; у остальных вершин эксцентриситет равен $+\infty$. В неорграфе минимальное значение эксцентриситета равно 2 (вершина v_2), максимальное — 4 (вершины v_0, v_4-v_7).

3.2. Лабораторная работа 2

3.2.1. Обход рёбер в ширину

BFS запускается из вершины, заданной пользователем. Программа выводит порядок посещения вершин, список рёбер дерева обхода и число итераций. Рёбра дерева выделяются на визуализации (рис. 10).

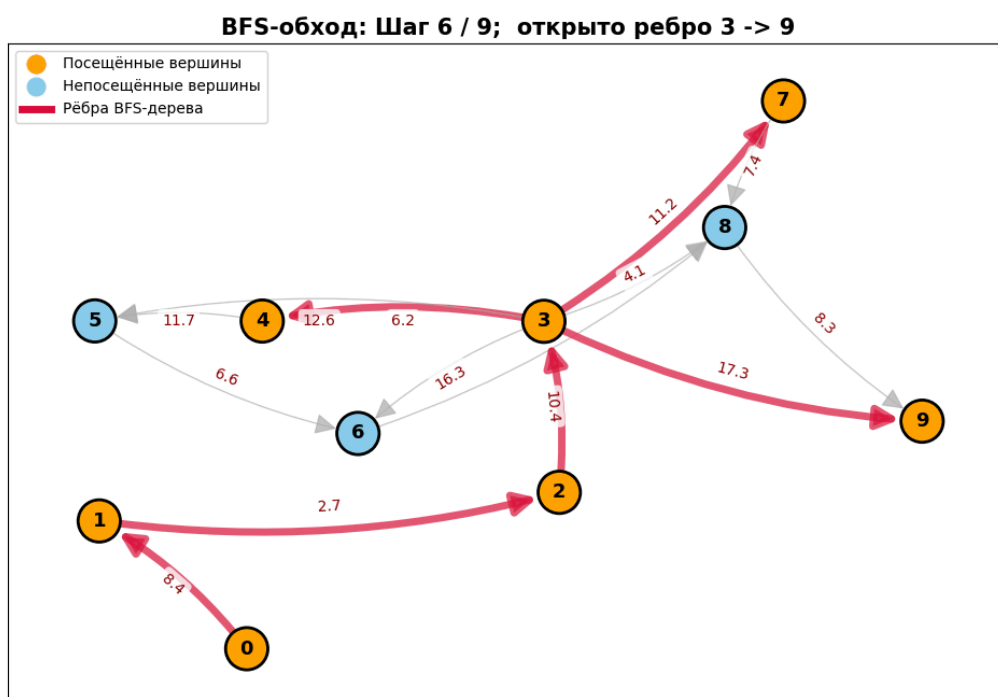


Рис. 10: Граф с выделенными рёбрами дерева BFS-обхода

3.2.2. Алгоритм Дейкстры

Пользователь вводит начальную и конечную вершины. Программа выводит вектор расстояний от источника до всех вершин, восстановленный путь как последовательность вершин и число итераций. Результат показан на рис. 11.

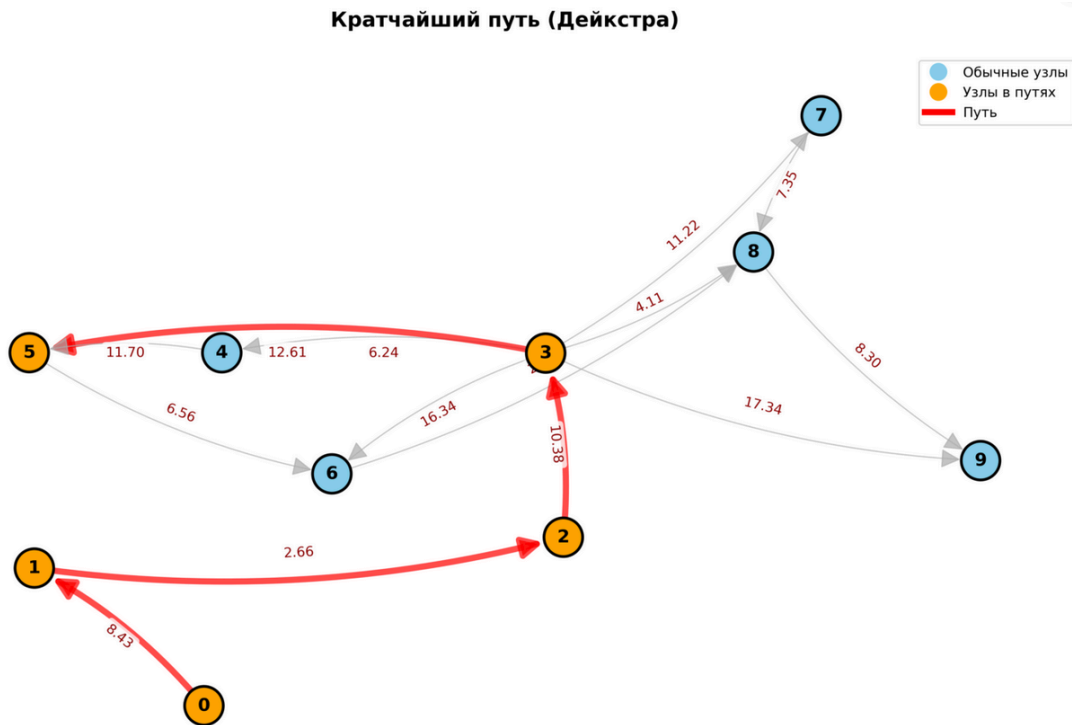


Рис. 11: Кратчайший путь по алгоритму Дейкстры

3.2.3. Сравнение алгоритмов

Оба алгоритма запускаются на одном графе с одной стартовой вершиной; подсчитываются обращения как к рёбрам, так и к внутренним структурам данных. Для теста был сгенерирован граф с 50 вершинами. Результат представлен в виде гистограммы на рис. 12.

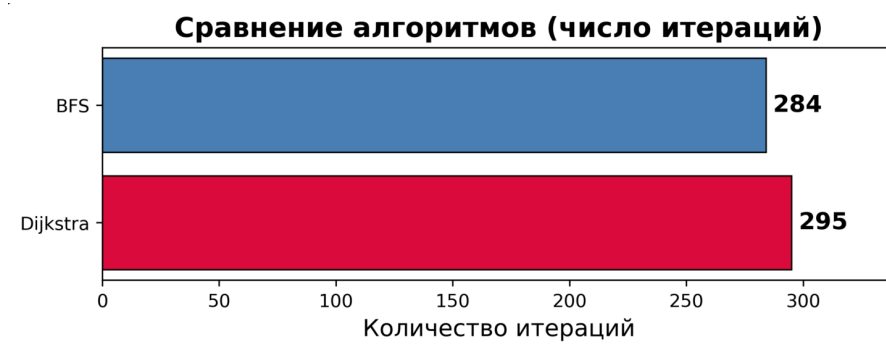


Рис. 12: Сравнение числа итераций BFS и алгоритма Дейкстры

3.3. Лабораторная работа 3

3.3.1. Формирование сети потоков

На основе ориентированного графа из предыдущих работ сформированы матрицы пропускных способностей C и стоимостей дуг W . Визуализация приведена на рис. 13 и 14.

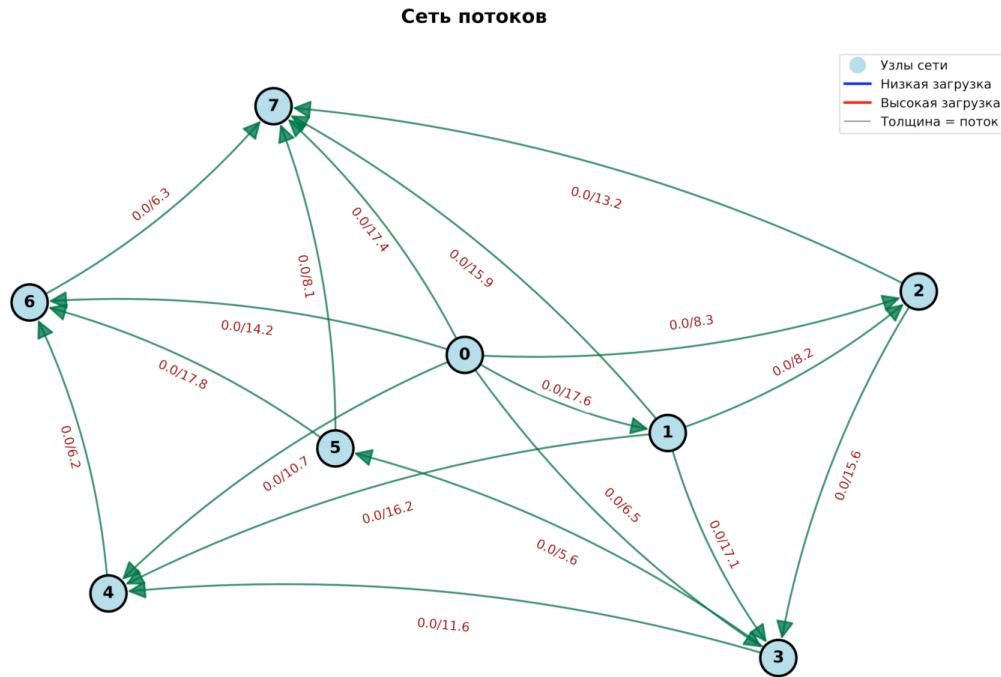


Рис. 13: Сеть потоков

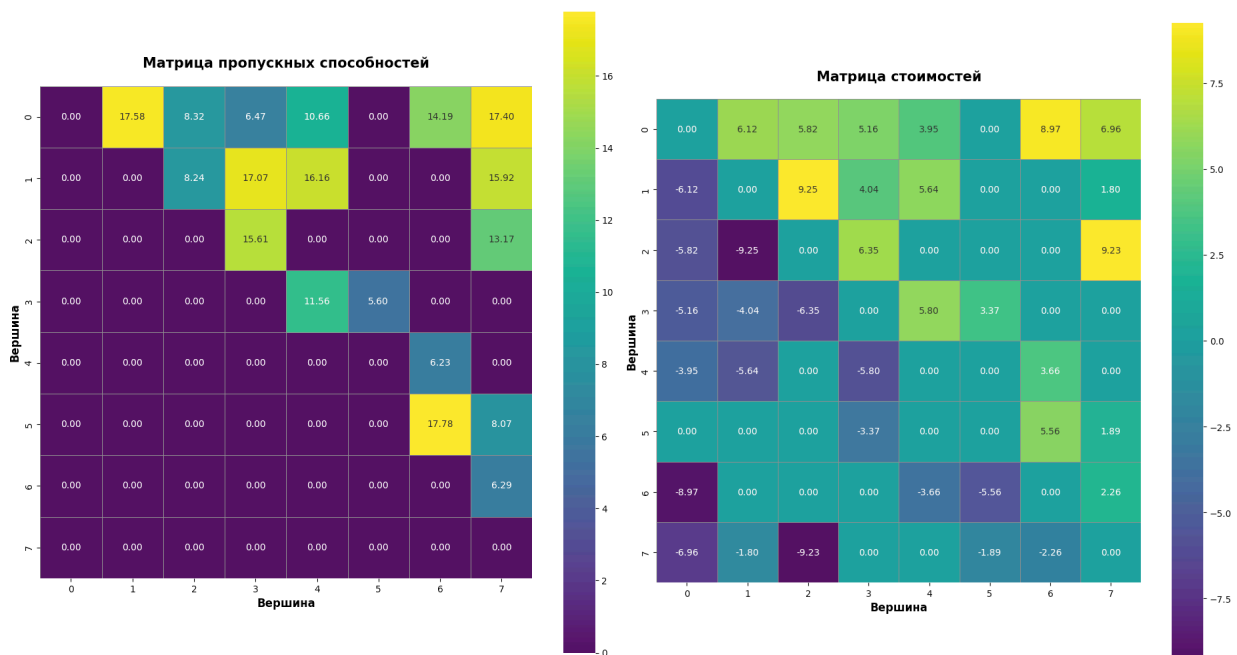


Рис. 14: Матрица пропускных способностей (слева) и матрица стоимостей (справа)

3.3.2. Максимальный поток

Алгоритм Форда–Фалкерсона итеративно находит увеличивающие пути в остаточной сети до их исчезновения. На рис. 15 показана финальная визуализация потока из вершины 0 в вершину 7.

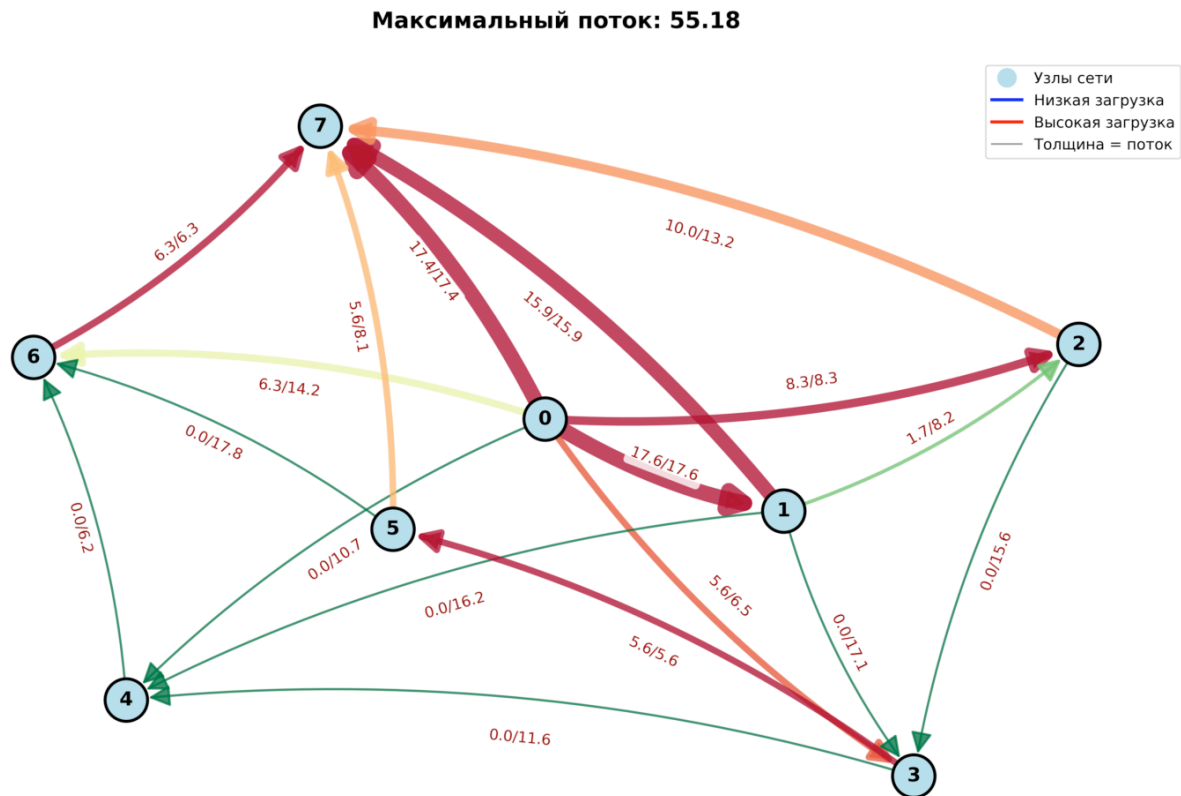


Рис. 15: Распределение максимального потока в транспортной сети

3.3.3. Поток минимальной стоимости

Целевая величина потока $F = \lfloor \frac{2}{3} \cdot F_{\max} \rfloor$ достигается последовательным поиском кратчайших путей по стоимости в остаточной сети алгоритмом Беллмана–Форда. Результат приведён на рис. 16 и 17.

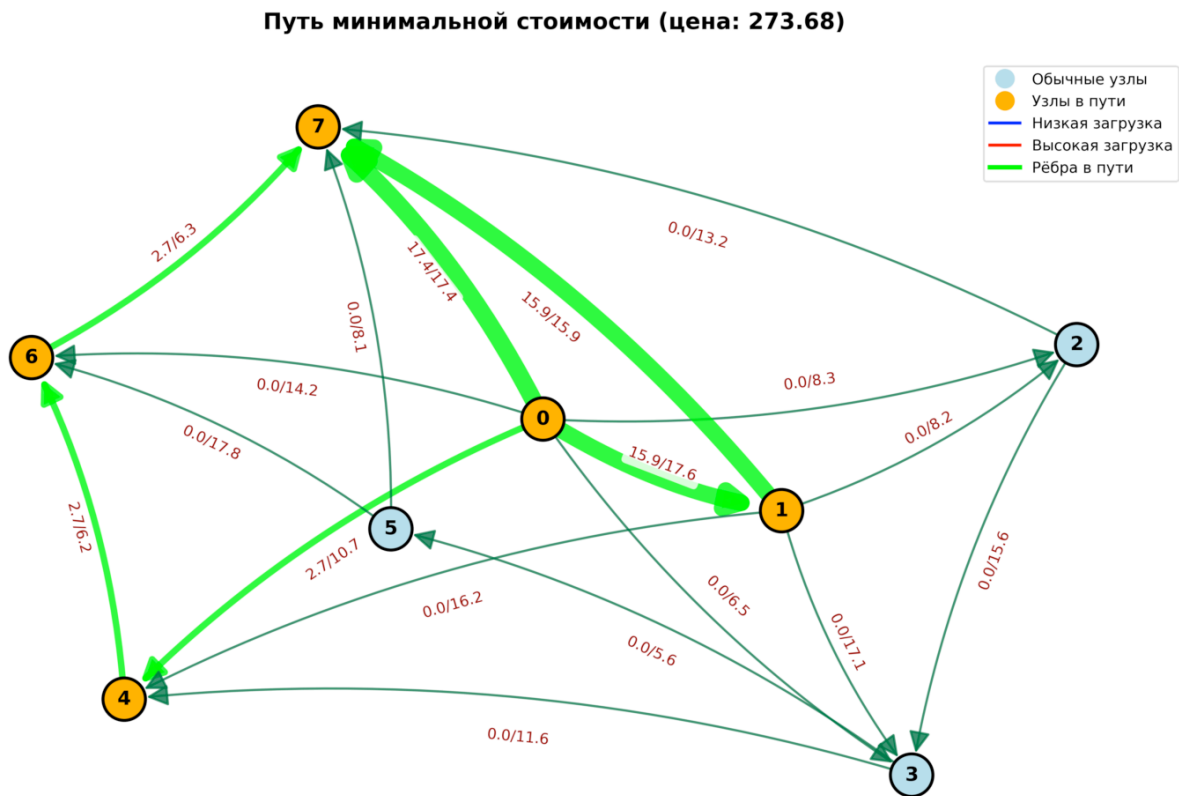


Рис. 16: Поток минимальной стоимости величиной $\lfloor 2/3 \cdot F_{\max} \rfloor$

```

=== Подробности по шагам ===
Шаг 1: путь 0 -> 7
Добавленный поток: 17.40
Стоимость пути за 1 ед.: 6.96
Приращение стоимости: 121.10
Накопленный поток: 17.40
Накопленная стоимость: 121.10
Шаг 2: путь 0 -> 1 -> 7
Добавленный поток: 15.92
Стоимость пути за 1 ед.: 7.92
Приращение стоимости: 126.07
Накопленный поток: 33.31
Накопленная стоимость: 247.17
Шаг 3: путь 0 -> 4 -> 6 -> 7
Добавленный поток: 2.69
Стоимость пути за 1 ед.: 9.86
Приращение стоимости: 26.51
Накопленный поток: 36.00
Накопленная стоимость: 273.68
  
```

Рис. 17: Консольный вывод: значение потока, суммарная стоимость, число итераций

3.4. Лабораторная работа 4

Граф и матрица его смежности показаны на рис. 18 и 19.

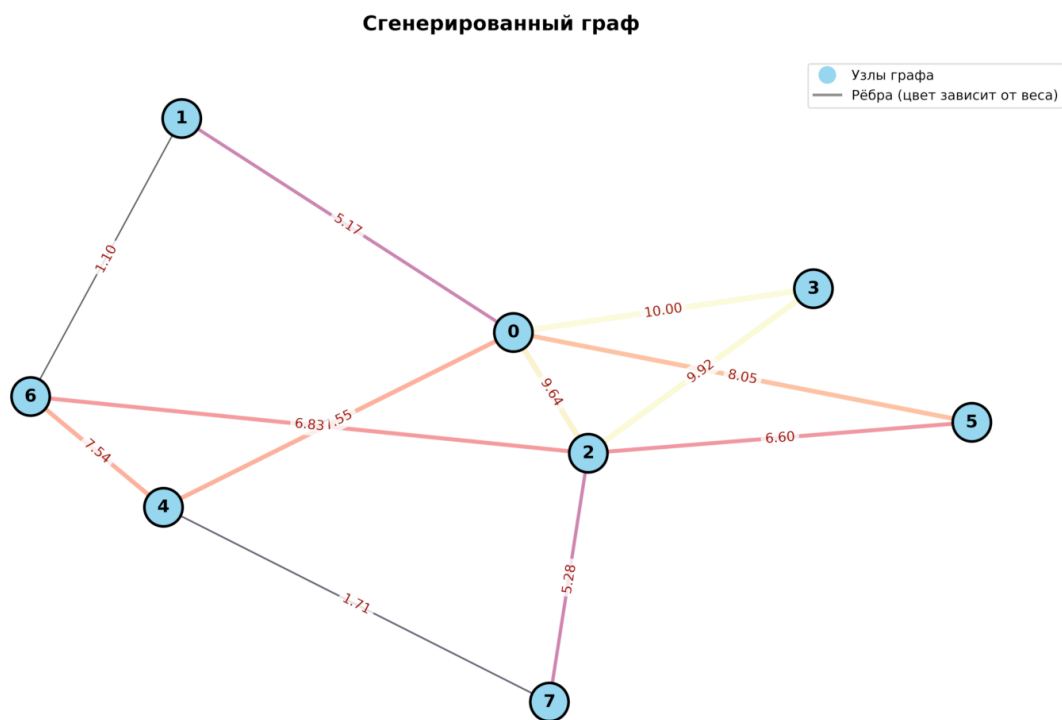


Рис. 18: Исходный неориентированный взвешенный граф

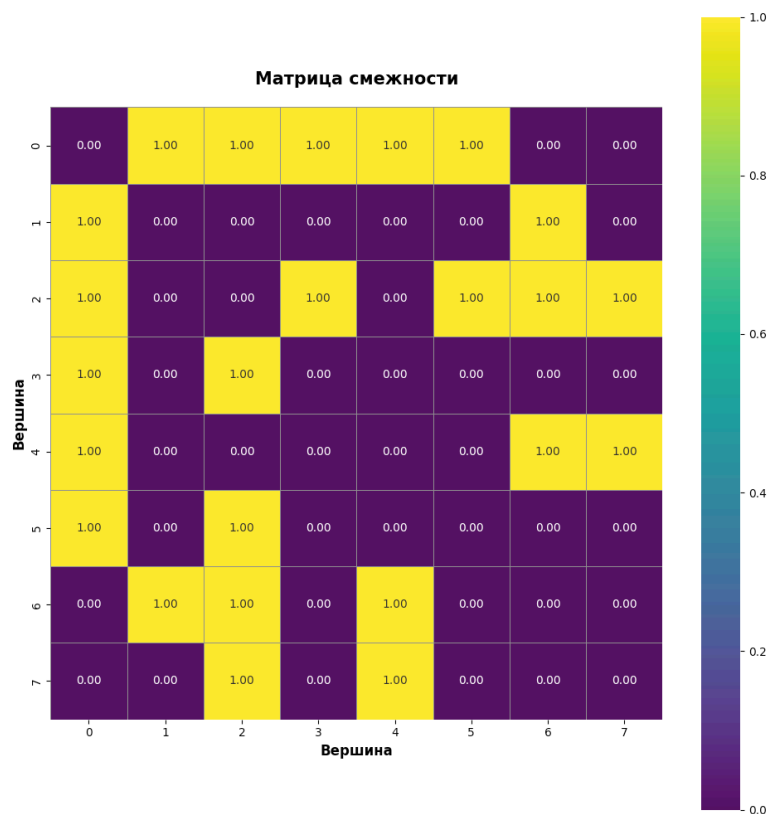


Рис. 19: Матрица смежности исходного графа

3.4.1. Матричная теорема Кирхгофа

Для данного графа число остовных деревьев составило 228.

3.4.2. Минимальный остов алгоритмом Борувки

Программа строит МОД и выводит число рёбер и суммарный вес. Визуализация приведена на рис. 20.

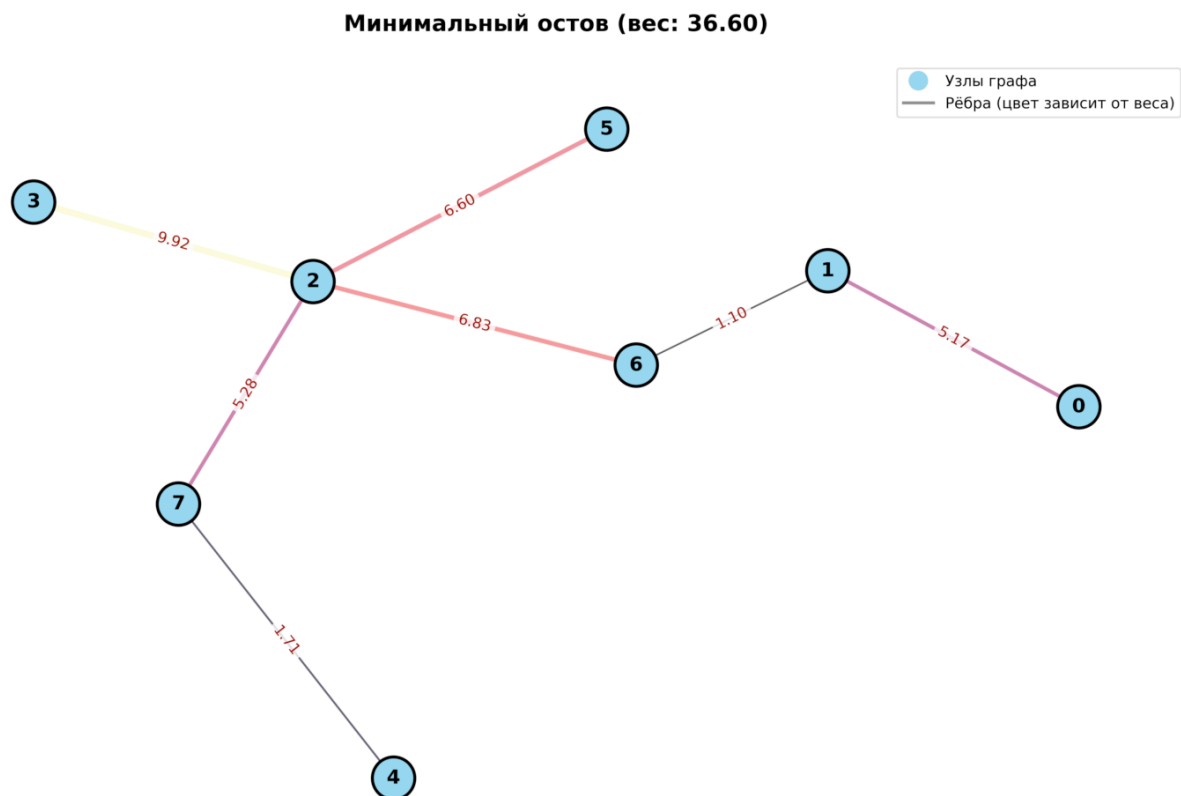


Рис. 20: Минимальное остовное дерево, построенное алгоритмом Борувки

3.4.3. Кодирование и декодирование кода Прюфера

Построенный МОД кодируется последовательностью Прюфера длины $n - 2$. Для графа из 8 вершин последовательность имеет длину 6:

1 6 2 7 2 2.

Декодированное дерево имеет тот же суммарный вес, что и исходный МОД, что подтверждает корректность кодирования. Визуализация восстановленного дерева приведена на рис. 21.

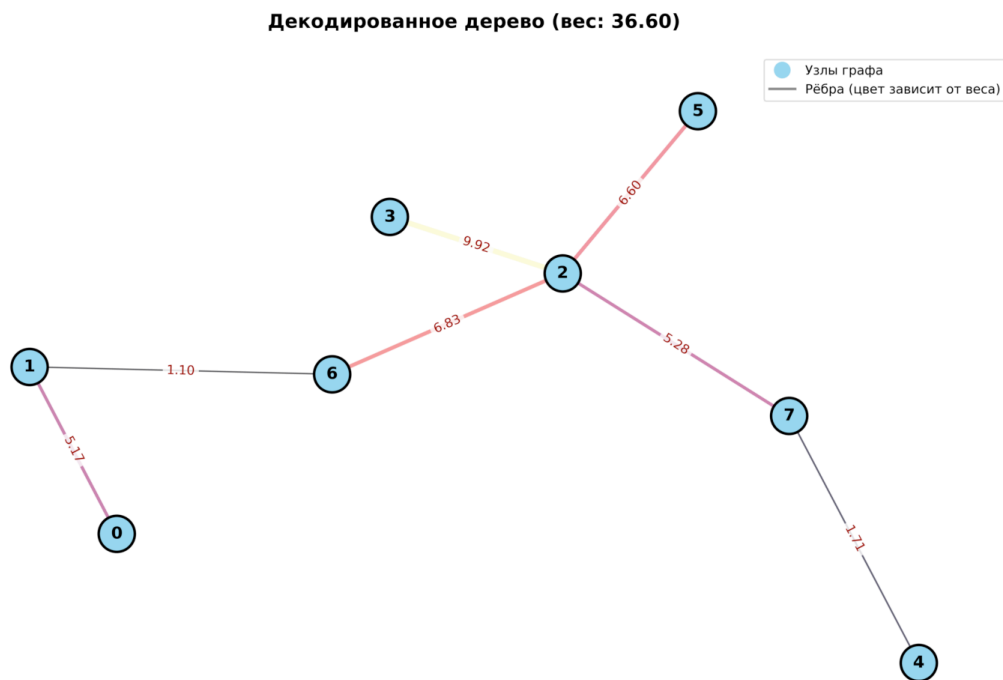


Рис. 21: Дерево, восстановленное из кода Прюфера

3.4.4. Минимальная раскраска графа

Пользователь выбирает граф для раскраски — исходный или МОД. Программа перебором с возвратом находит хроматическое число $\chi(G)$ и выводит цвет каждой вершины. Визуализация с оптимальной раскраской приведена на рис. 22.

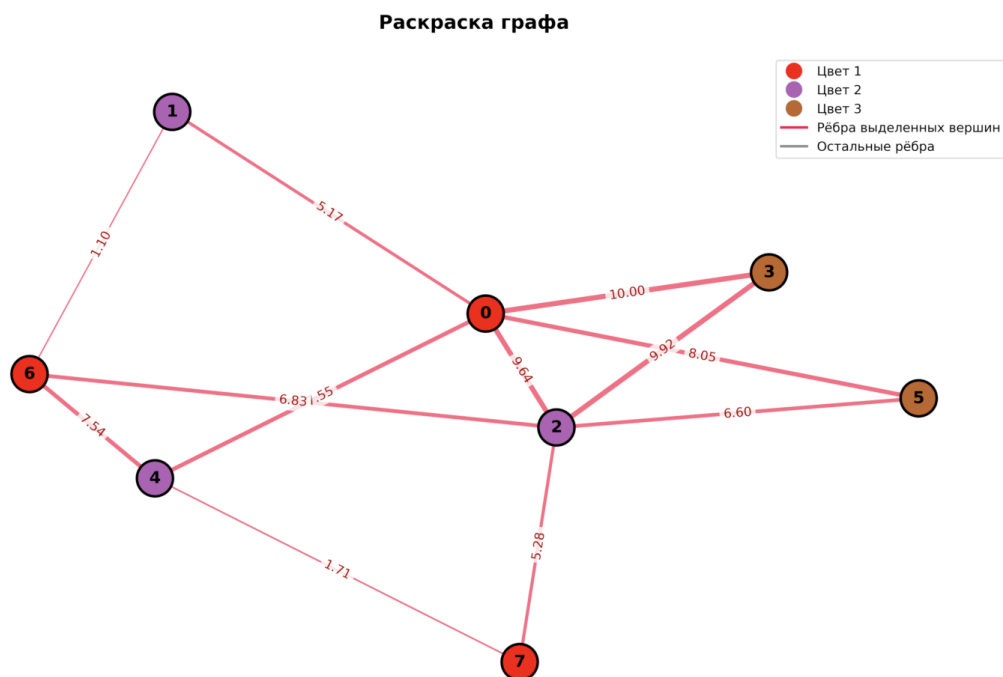


Рис. 22: Граф с минимальной раскраской вершин

3.5. Лабораторная работа 5

3.5.1. Проверка эйлеровости и построение эйлерова цикла

Достроенный граф до эйлерова с визуализированным эйлеровым циклом на нем показан на рис. 23 и 24.



Рис. 23: Граф с выделенным эйлеровым циклом; добавленные рёбра отмечены отдельным цветом

```
[INFO] Граф не эйлеров, вершин с нечётной степенью: 4 — выполняется достройка
[EULER][DEBUG] makeEulerian: start
[EULER][DEBUG] non-zero components: 1
[EULER][DEBUG] iteration 1, odd vertices: count=4 [0, 2, 4, 6]
[EULER][DEBUG] add edge 0--6 [added]
[EULER][DEBUG] add edge 2--4 [added]
[EULER][DEBUG] pairs selected: 2, edges actually added: 2
[EULER][DEBUG] iteration 2, odd vertices: count=0 []
[EULER][DEBUG] makeEulerian: end, odd vertices: count=0 []
[INFO] Добавлено рёбер: 2
+ 0 -- 6
+ 2 -- 4
[OK] После модификации граф эйлеровый
[OK] Найден маршрут, длина: 15 (замкнутый цикл)
[PATH] 0 -> 1 -> 6 -> 0 -> 2 -> 5 -> 0 -> 4 -> 7 -> 2 -> 6 -> 4 -> 2 -> 3 -> 0
```

Рис. 24: Консольный вывод: результат проверки, добавленные рёбра и маршрут

3.5.2. Фундаментальная система разрезов

Программа строит МОД алгоритмом Борувки и вычисляет фундаментальный разрез для каждого ребра остова. Визуализация МОД приведена на рис. 25; визуальный вывод ФСР — на рис. 26.

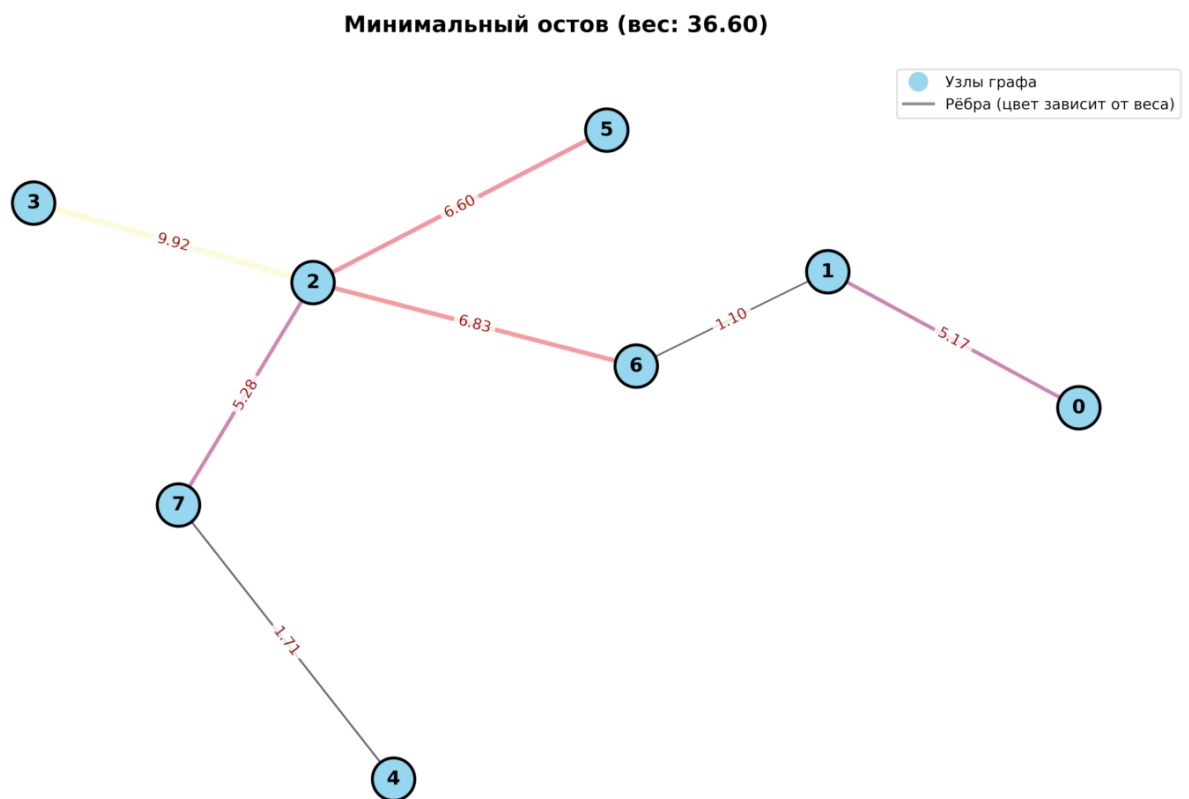


Рис. 25: Минимальный остов, построенный алгоритмом Борувки

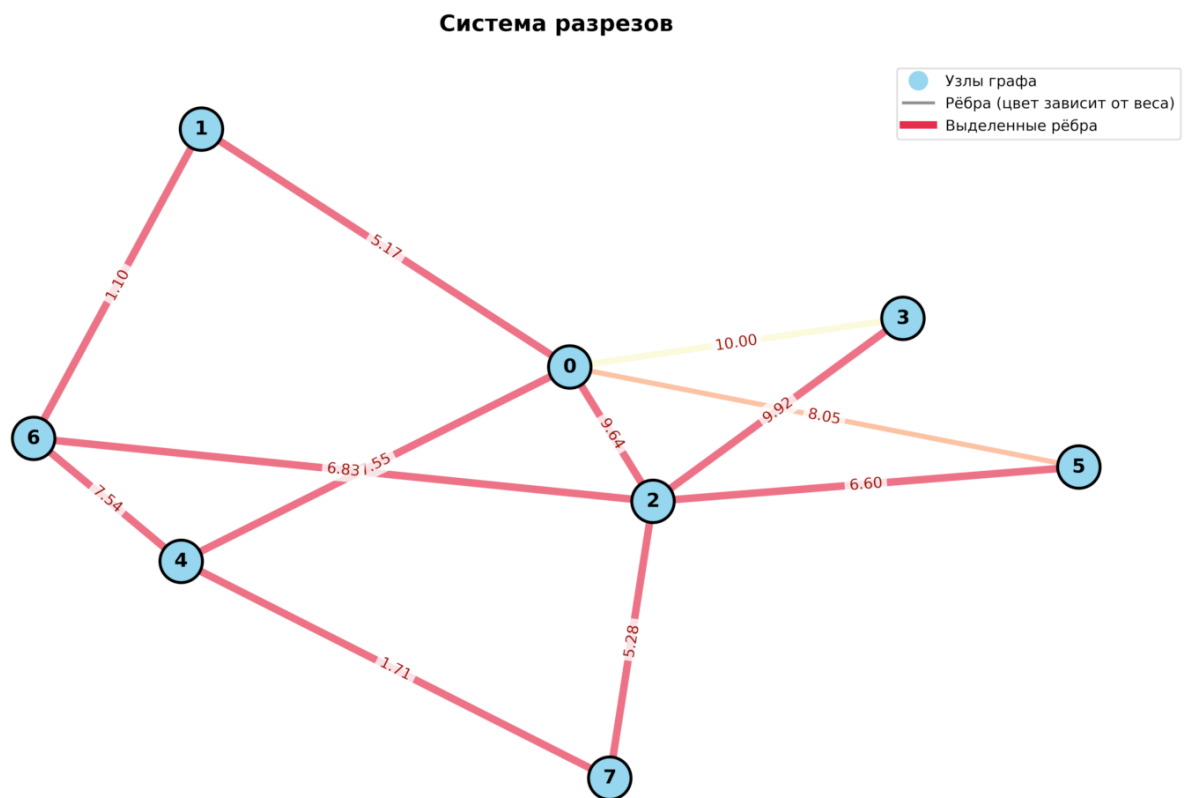


Рис. 26: Граф: фундаментальная система разрезов

Фундаментальная система разрезов:

- 0) Удалено $(2, 6) \Rightarrow A = \{2, 3, 4, 5, 7\}, B = \{0, 1, 6\}$.
Разрез: $\{(0, 2), (0, 3), (0, 4), (0, 5), (2, 6), (4, 6)\}$.
- 1) Удалено $(2, 7) \Rightarrow A = \{0, 1, 2, 3, 5, 6\}, B = \{4, 7\}$.
Разрез: $\{(0, 4), (2, 7), (4, 6)\}$.
- 2) Удалено $(2, 3) \Rightarrow A = \{0, 1, 2, 4, 5, 6, 7\}, B = \{3\}$.
Разрез: $\{(0, 3), (2, 3)\}$.
- 3) Удалено $(2, 5) \Rightarrow A = \{0, 1, 2, 3, 4, 6, 7\}, B = \{5\}$.
Разрез: $\{(0, 5), (2, 5)\}$.
- 4) Удалено $(1, 6) \Rightarrow A = \{0, 1\}, B = \{2, 3, 4, 5, 6, 7\}$.
Разрез: $\{(0, 2), (0, 3), (0, 4), (0, 5), (1, 6)\}$.
- 5) Удалено $(0, 1) \Rightarrow A = \{0\}, B = \{1, 2, 3, 4, 5, 6, 7\}$.
Разрез: $\{(0, 1), (0, 2), (0, 3), (0, 4), (0, 5)\}$.
- 6) Удалено $(4, 7) \Rightarrow A = \{4\}, B = \{0, 1, 2, 3, 5, 6, 7\}$.
Разрез: $\{(0, 4), (4, 6), (4, 7)\}$.

Симметрическая разность по всем разрезам:

$\{(0, 1), (0, 2), (0, 4), (1, 6), (2, 3), (2, 5), (2, 6), (2, 7), (4, 6), (4, 7)\}$.

При передаче пустого множества алгоритм возвращает исходную ФСР.

3.5.3. Симметрическая разность подмножества разрезов

Программа позволяет выбрать произвольное подмножество базовых разрезов (для примера выбраны разрезы 0, 2 и 4) и вычисляет их симметрическую разность. Результирующий разрез выводится в консоль и выделяется цветом на графе (рис. 27 и 28).

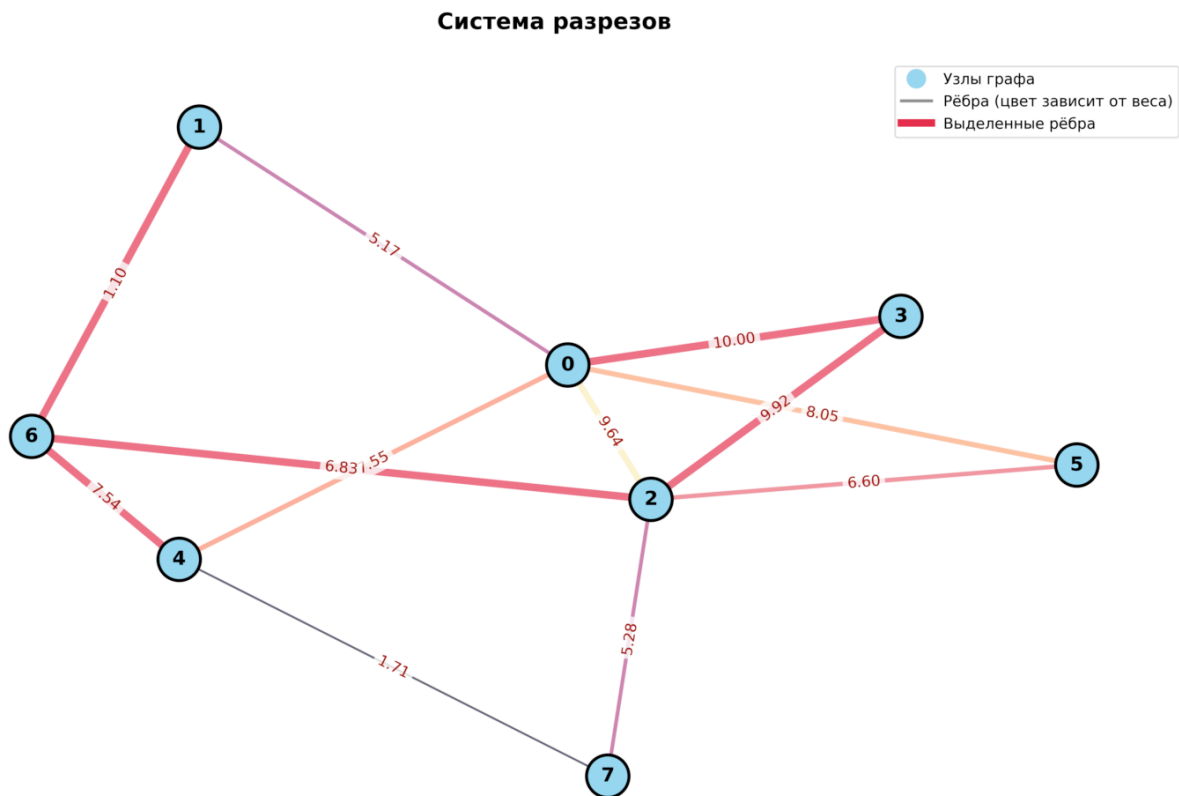


Рис. 27: Граф с выделенными рёбрами симметрической разности

```

Выбранные разрезы:
0) Ребро остова (2, 6) | Рёбра разреза: { (0, 2) (0, 3) (0, 4) (0, 5) (2, 6) (4, 6) }
2) Ребро остова (2, 3) | Рёбра разреза: { (0, 3) (2, 3) }
4) Ребро остова (1, 6) | Рёбра разреза: { (0, 2) (0, 3) (0, 4) (0, 5) (1, 6) }

Процесс XOR:
Вхождения рёбер в выбранные разрезы:
(1, 6): 1 раз (2, 3): 1 раз (4, 6): 1 раз (0, 5): 2 раз (0, 4): 2 раз (2, 6): 1 раз (0, 3): 3 раз (0, 2): 2 раз
[RESULT] Симметрическая разность (индексы: 0 2 4 ): { (0, 3) (1, 6) (2, 3) (2, 6) (4, 6) }

```

Рис. 28: Консольный вывод: вхождения рёбер и результирующий разрез

Заключение

В рамках лабораторных работ 1–5 реализован полный цикл работы с графом: генерация связного ациклического графа по распределению Рэлея–Райса с softmax-нормировкой степеней, вычисление характеристик (эксцентриситеты, радиус, диаметр, центр), построение матриц минимальных и максимальных маршрутов длины k методом Шимбелла, перечисление всех простых путей между двумя вершинами, обход рёбер графа в ширину, нахождение кратчайшего пути алгоритмом Дейкстры с выводом вектора расстояний и сравнение алгоритмов по числу итераций. Исходный код реализации всех лабораторных работ размещён в открытом github-репозитории [6].

В лабораторной работе 3 на основе порождённого графа формируется транспортная сеть: строятся матрицы пропускных способностей и стоимостей дуг, находится максимальный поток и поток минимальной стоимости величиной $\lfloor \frac{2}{3} \cdot F_{\max} \rfloor$ методом последовательных кратчайших путей на базе алгоритма Беллмана–Форда.

В лабораторной работе 4 для неориентированного графа вычисляется число остовных деревьев по матричной теореме Кирхгофа, строится минимальный остов алгоритмом Борувки, полученное дерево кодируется и декодируется кодом Прюфера с сохранением весов рёбер, а также находится минимальная раскраска графа методом перебора с возвратом.

В лабораторной работе 5 реализована проверка графа на эйлеровость с автоматической достройкой графа и нахождением эйлерова цикла алгоритмом Иерхольцера. Также построена фундаментальная система разрезов (ФСР) на базе остова Борувки с вычислением симметрической разности для любого выбранного подмножества разрезов.

Все пункты заданий выполнены. Граф из лабораторной 1 переиспользуется в последующих работах без регенерации. Результаты каждого шага сохраняются в текстовые файлы и передаются в Python-скрипты для визуализации без перезапуска исполняемого файла.

Сильные стороны реализации

Архитектура разделена на независимые слои: граф не знает о генераторе, генератор не знает об алгоритмах, алгоритмы не знают о визуализации. Это позволяет заменять любой компонент без изменения остальных.

Шаблонный GraphBase даёт одну кодовую базу для графа и сети потоков за счёт статического полиморфизма. Шаблонный генератор с лямбдами переиспользуется для обоих типов графов и любого закона генерации весов без дублирования алгоритма построения.

`std::optional` в матрицах Шимбелла и в интерфейсе графа исключает специальные значения и делает семантику «ребра нет» и «пути нет» явной на уровне типов.

Наличие веб-интерфейса с полной визуализацией и анимацией алгоритмов на графах.

Слабые стороны реализации

Метод Шимбелла работает за $O(k \cdot n^3)$ и хранит две матрицы $n \times n$ в памяти.

Поиск всех простых путей имеет факториальную сложность. На разреженных ациклических графах это допустимо, но никакой защиты при больших входных данных не предусмотрено.

Минимальная раскраска реализована полным перебором с возвратом со сложностью $O(n \cdot k^n)$ в худшем случае — алгоритм применим лишь на небольших графах и не масштабируется на плотные графы с большим хроматическим числом.

Передача данных между C++ и Python реализована через текстовые файлы: C++ записывает результат в `assets/txt/`, Python читает его при запуске скрипта. Формат намеренно простой для учебной задачи. В production-окружении такое взаимодействие между процессами заменяется брокерами сообщений (Kafka, RabbitMQ) или очередями задач (Celery, ZeroMQ) — они обеспечивают типизированные сообщения, подтверждение доставки и горизонтальное масштабирование.

Масштабирование

Замена файлового обмена на брокер сообщений развяжет процессы по времени, позволит запускать несколько воркеров визуализации параллельно и добавит гарантии доставки.

Переход на WebSocket позволит C++ передавать результат напрямую в браузер: визуализация появится сразу по завершении вычислений без перезагрузки страницы.

Замена статичных PNG на интерактивные графы через Plotly — с возможностью масштабирования, выделения вершин и фильтрации рёбер по весу прямо в браузере.

Параллелизация алгоритмической части: метод Шимбелла считает n независимых строк матрицы на каждом шаге, поиск всех путей разбивается по стартовым рёбрам, а итерации алгоритма.

Покрывание тестами GTest граничных случаев генератора, корректности матриц Шимбелла, полноты поиска путей, инвариантности веса при кодировании и декодировании кода Прюфера и правильности раскраски.

Список литературы

- [1] Секция «Телематика» [Электронный ресурс]. — URL: <https://tema.spbstu.ru/dismath/> (дата обращения: 18.05.2025).
- [2] Новиков Ф. А. *Дискретная математика для программистов*. — 3-е изд. — СПб.: Питер Пресс, 2009. — 384 с.
- [3] Вадзинский Р. Н. *Справочник по вероятностным распределениям*. — СПб.: Наука, 2001. — 295 с.
- [4] Асанов М. О., Баранский В. А., Расин В. В. *Дискретная математика: Графы, матроиды, алгоритмы*. — 2-е изд. — СПб.: Лань, 2010. — 368 с.
- [5] Эриксон Дж. *Алгоритмы*. — М.: МЦНМО, 2024.
- [6] Репозиторий GitHub с исходным кодом проекта [Электронный ресурс]. — URL: <https://github.com/ivanchetvergov/poly/tree/master/sem4/graph> (дата обращения: 07.05.2026).